The Thiele Machine

The worked machine

Devon Thiele

What this is

The book *The Thiele Machine: No Free Insight* defines the model and argues about it. This companion builds one machine that meets the definition and computes everything a Turing machine can, and then takes it apart. It is an example. It is not the Thiele Machine, and nothing in the book depends on its details.

The machine is two counters and a line number for its flat part, and a record whose moves check a claim, commit to a passed check and certify it, in that order and at one unit each. Its flag is a record in the book's sense: it never falls, and a three-move run from a clean start raises it, so it is a Thiele machine. Beside it are the universal programs U and U_P, a host that runs other machines as guests, and the ways machines like it combine.

Numbered results, sections and definitions cited here that aren't in this companion are in the book. Copyright 2025–2026 Devon Thiele. Licensed under Creative Commons Attribution-ShareAlike 4.0.

Contents

1 The earned-chain counter example 4
1.1 What the model asks for, and the example that meets it 4
1.2 Where the counters come from 7
1.3 The example machine, piece by piece 9
1.4 It computes everything two counters can 20
1.5 It's a Thiele machine, and the flag is a latch 23

1.6 Commitments are earned 25
1.7 What a passing check means 28
1.8 Nothing gets forged 31
1.9 The window can't see the record 32
1.10 The exact price is three 37
1.11 The machine versus the program it runs 39
1.12 The small machine's merges 41
1.13 Reading the instruction rules 42
1.14 What this chapter adds up to 46

2 How far it reaches 47

3 The universal machines 49
3.1 One machine that runs them all 50
3.2 Every computably presented machine, on one host 58

4 Built from parts 65
4.1 Side by side 65
4.2 One after the other 68
4.3 Connecting records and writers 69
4.4 One inside another 71
4.5 What this adds up to 74

5 The toll on this machine 75

6 No bare verifier on this machine 79

7 Scope of these results 81

8 Words for this machine 83

9 The premises this machine adds 85

10 What's mine about this machine 86
10.1 An example machine that meets the definition 86
10.2 The universal host for the example 87

11 The claims about the worked machine 89

CHAPTER 1

The earned-chain counter example

The chapters before this one never built a machine you could program. They said "any states, any steps," and whenever a picture was needed, a board or some chairs or a cup stood in. Now I have to build a machine, and the first question is about the one I built before it. I built a big one first, on the way here (the Afterword, Section 23, tells that story), with dozens of instructions, some picked for reasons and some picked because I had to pick something, and theorems held of it. What does that show? That the model works, or that I'm good at stacking instructions until the proofs go through? A big machine is a wonderful place to hide a mistake. It's a garage so full of stuff that you could lose a car in it.

So the example machine in this book is a small one. I want the state and every way of changing it where you can inspect them. There are six instruction kinds, and you can follow how a check stays tied to an object, how a change makes it stale, and how the next move uses that relation. Each kind takes operands, a counter or a property or a line to jump to, so there are unboundedly many instructions. It's one way of building the model, not the only one, and the theorems will say which facts hang on these particular choices. I built it by throwing things out until nothing more would go. The smallest Thiele machine is the latch, and that one is proved (Theorem 2.3). This is the smallest I could build that also earns its flag, and for that I haven't got a proof that nothing smaller works, and I'm not claiming one.

1.1 What the model asks for, and the example that meets it

The model asks for states, moves, a step, a price on each move and a record in an order, with the toll, a record that never falls, and some run from a named start that raises it. Before any of its parts, I'll give the example that meets it in three sentences. Its record has two places, a flag that is the blank cell, and its flat part is two counters and a jump, the part a window on the counters shows. Raising the flag settles it for good, a merge in the record's part of the state (flag down and flag up land on the

same state), which is why the step that raises it is charged once merges in that part are priced (Theorem 5.1, in Part I). And the order rule, check, then commit, then certify, is what makes a raised flag mean something.

Parts I and II define the model by five parts, the toll, a record that never falls, and a run that raises it. The example meets them with the smallest *record* there is, a flag that reads no or yes and, once yes, stays, which is the smallest record there is. Here yes means "certified," because that's the event I picked (Section 17, later in this part, asks whether anything outside the rules picks it out). It adds a *toll*: the step that raises the flag costs something (Section 2). And it adds a *ledger*, the running total of those costs, a whole number that only climbs (Section 8). And Section 12 proved the thing this whole chapter stands on: over any base at all, a base-driven record is a *latch*, a bit that switches on at an event the base decides and never switches off (Observation 12.1). The record watches the base. It doesn't compute anything the base couldn't compute without it.

Push on that for a second, because it cuts the job in half. If the record adds no computing power, then all the computing power comes from the base, and the question "how small can the machine be" splits into two questions that don't touch each other. For the base: what's the smallest thing that can compute everything there is to compute? For the record: what's the smallest set of pieces that makes the flag *mean* something?

There's one honest asterisk on "don't touch," because the machine below makes it matter. What's blind is the flag and the ledger: the base never reads either of them. But the pieces that make the flag mean something (a table of checked facts, a slot for a commitment, a trap that stops a refused run) have to sit where the step can read them, which is in the base, and they do steer the run: a failed check stops a program dead. The latch theorem holds wherever you draw the line between base and record, which is exactly why it can't tell you where the meaning lives. On this machine the meaning lives on the base side of that line, in the bookkeeping, and the flag on top reports it. That line has to sit at the core: the step reads the table, the channel and the trap, so a map that hid any of them wouldn't move in step. The record-layer merge of Part I draws a lower line, with only the line number and the two counters below it, and what the premise charges moves with that line. Drawn there, all three record acts merge in the record's part and the premise charges each of them; drawn at the core, it charges only CERTIFY.

The record question, the second of those two, is the one Part II left to the machine, on purpose. Section 11 said that testing a claim and committing to it are two different acts, and that nothing in the five parts of the model makes the second wait for the first. Read Section 2's definition again with that in mind: a machine where you pay one unit and raise the flag over nothing in particular obeys every rule in it. That machine is honest about the price and silent about what was bought:

it's a receipt for nothing. The toll alone can't fix that, because the toll is a rule about *cost*, and "was this earned" is a question about *order*: did a check happen, of this claim, before the commitment, with nothing changing underneath in between?

This machine is my attempt to make the order part of the machine, in the smallest place I could find to do it. I'll show you the pieces, then the proofs, then the places it falls short, which there are, and which I'd rather you hear from me.

The blown fuse cannot be hidden from every future move

Blow the fuse and pull the lever. Nothing moves. Replace the fuse, start from the same counter values, and that lever can turn the gears. The fuse decides what the next pull does, whether the camera can see it or not.

That is what a failed CHECK does here. It traps the core, and every free instruction then leaves the whole state alone. Suppose I hide the failed check from the camera and still demand that the next picture depend only on the present picture and the free move. The live and trapped pictures agree. The camera's rule must give them the same next picture. Since the trapped box stays still, the projected live box has to stay still too.

□ **Theorem 1.1** (A globally check-silent native shadow is static)**.** Let π be any projection from native full states, with any target type. Suppose every CHECK preserves π, and each zero-cost instruction i has an autonomous projected rule h_i, so $\pi(\mathrm{exec}(s,i)) = h_i(\pi(s))$ for every state. Then $h_i(\pi(s)) = \pi(s)$ for every s, i. If π is onto, each h_i is identity everywhere. The conclusion also holds on the reachable states from a fixed start when those same laws hold throughout that reachable domain.

Proof. Stand at any native state s and read counter A. Ask CHECK whether that counter is at least one greater than the number just read. It is not, so this check leaves a trapped state t. In symbols, the chosen question is $A \geq A(s) + 1$.

By the proposed silence rule, the failed check has not changed the picture: $\pi(t) = \pi(s)$. Now choose any free instruction i. At the trapped state it changes nothing at all. The camera's rule h_i must therefore leave that picture alone. But it is also the picture of s, and h_i receives only the picture, not a choice between the two boxes. It must leave s's picture alone too. Written as one chain,

$$h_i(\pi(s)) = h_i(\pi(t)) = \pi(\mathrm{exec}(t,i)) = \pi(t) = \pi(s).$$

You could begin at any s and choose any free i, so every free projected move is still on the whole image. If every base picture occurs, that is the whole base. I did choose a different failing question for different counter values; the premise allows

every CHECK at every state, so it allows each of those choices. Finally, if s was reached from the fixed start, append this CHECK to that run to reach t. Both boxes remain inside the reachable domain, and the same argument applies there. ■

The fuse belongs with the gears that read it. The certificate and ledger can sit beside them unread. The later slice follows the computing moves while the machine is live; a failed check takes it out of that working domain. That is where the line goes on this machine. [1]

1.2 Where the counters come from

The base first, and the base isn't mine. It's Marvin Minsky's. In a 1961 paper in the *Annals of Mathematics*, and then in his 1967 book *Computation: Finite and Infinite Machines*, Minsky studied a machine so bare it's almost a joke. Picture a few bins of pebbles and a numbered list of instructions. There are only two kinds of instruction: put a pebble in bin c and go to the next line; or take a pebble out of bin c if there's one to take, and go one way if there was and another way if the bin was empty. There's nothing else: no memory but the bins, no tape, no arithmetic beyond plus one and minus one. When the line you're on has no instruction, the machine stops.

What Minsky proved is that two bins are enough to compute anything a Turing machine can compute, as long as you're willing to pack the input in a strange way. The trick is to keep many numbers inside one number, as the powers of different primes ($2^x3^y5^z$ holds three numbers in one), and to do arithmetic on the exponents by multiplying and dividing with the second bin as scratch space. Packing numbers that way can turn a modest input into an enormous pile of pebbles, handled one at a time, so it's no way to build a fast machine. It's a way to settle a question: with that packing, the two bins can do every Turing computation. What the bins give me here is a base whose ordinary moves you can see all of. Each of the earlier results says which properties of a base it needs, and the earned layer carries over to other bases once you've named what they have to supply.

Who cares? Minsky did, because the question then was where the edge of computation sits and how little machinery reaches it. You should, because it means the base of this chapter's machine hides nothing: if it can compute everything with plus one, minus one and a jump, then nothing about the record can be smuggling computing power in through some instruction I made fancy. For this construction I use Minsky's two-counter halting result and the counter constructions of Forster, Larchey-Wendling and their coauthors. Their reduction is theirs; the step-for-step connection to this machine is the part I have to establish.

[1] A view that exposes refusal, or a machine with a different failure rule, would supply a different interface. The obstruction concerns the native failure semantics and a globally CHECK-silent autonomous projection.

And the pebbles aren't only my picture. They were somebody's actual machine, the same year. In one issue of the Canadian Mathematical Bulletin, September 1961, Z. A. Melzak, a mathematician at the University of British Columbia, described a computer made of distinguishable holes and an unlimited supply of pebbles, with one three-place operation, and a few pages later Joachim Lambek, a mathematician at McGill University, trimmed it to two moves: put a pebble in hole X; take one out of hole X if there's one to take. He called it an *infinite abacus*, and early on he writes "Our method is probably not new," which is a hard sentence to beat. He also tells anybody who distrusts infinity not to worry about all those holes, since any one computation only ever uses finitely many of them. Is that two inventions or one? Lambek says his was a modification of Melzak's, so call it one and a half, with Minsky's as a separate find the same year. And then Lambek went somewhere you wouldn't guess: in 1986, with Philip Scott, a logician who had come to McGill to work with him and later taught at the University of Ottawa, he wrote a book whose first part, as the publisher describes it, shows that typed lambda calculi and a certain kind of category are essentially the same thing. He went from pebbles to category theory in one career, twenty-five years end to end. Does any of it change the argument? No. The machine uses Minsky's version and Minsky's name. But a bare machine that several people reach for in the same year looks like a shape that was sitting there. It doesn't look much like anybody's invention.

The title of Minsky's 1961 paper is the clue I can't leave alone. It's about "Post's problem of tag." Who's Post, and what's tag? Emil Post was a logician, a fresh PhD from Columbia, who in 1921, on a fellowship at Princeton, was playing with about the simplest rewriting game there is. Take a word of 0s and 1s. Look at its first letter, chop the first three letters off the front, and stick something on the end: `00` if the letter you looked at was 0, `1101` if it was 1. Repeat, and stop when fewer than three letters are left. Try it at the table. Start from `1000`: you get `01101`, then `0100`, then `000`, then `00`, and it stops. Start from `100`: you get `1101`, `11101`, `011101`, `10100`, `001101`, and then `10100` again, and it goes round forever. Post wanted a rule that tells you, for every starting word, which of those happens. He worked at it for months and later called it hopeless, and as far as I can tell it still is: a hundred years on, nobody knows whether that one little game, with that one rule, can be decided. Stephen Wolfram, the physicist who built Mathematica, wrote a 2021 article on it that tells the Princeton story, and it's where I'd start. In his 1961 paper Minsky proved that tag games in general can compute anything, which is why Post's name is on it. It didn't settle Post's own game. And the chase goes one step further. In 2008 Liesbeth De Mol, a historian and philosopher of computing who wrote her doctoral work on tag systems, turned the famous $3n + 1$ problem (halve an even number, triple an odd one and add one, and ask whether you always get down to 1) into a small tag game of the same kind. So is Post's game secretly as hard as everything, or

just stubborn? I can't tell you, and as far as the published work goes, nobody can yet.

None of that changes the machine below. It changes the picture. The two-counter machine came out of a hunt for the smallest thing that can't be predicted, and the smallest things are exactly where the open questions are still sitting.

There's one word clash to settle before it bites. Minsky's machine is usually called a *counter machine*, and its bins are *counters*. Section 8 also had a counter: the ledger μ, the number that only climbs. It's the same word for two completely different things, and I'd love to say I set that up on purpose. In this chapter, *counter* means one of Minsky's two bins, called A and B, and the climbing total is always *the ledger* μ.

1.3 The example machine, piece by piece

I'll go through the whole state a piece at a time, in words first.

- Two counters, A and B. Each holds a whole number, with no upper limit.
- A *version* for each counter, also a whole number with no limit. It starts at 0 and goes up by one every time its counter is written. Think of it as the counter's edition number: a new value is a new edition.
- A *line number*, which instruction comes next, counting from 1 (more on that choice just below).
- A *fact table*: a list of facts. A *fact* is three things stuck together: a property, a counter, and a version. "A is even, as of edition 3" is a fact. The table has room for sixteen. Why sixteen? It's a pick, with no derivation behind it, and what the pick gives up is set out in the subsection near the end of this chapter on what the machine doesn't do.
- A *channel*: either empty, or holding one fact. This is where a commitment sits.
- A *trap*: one bit, down at the start. When it goes up, the machine has refused something, and it stays up.

Counting lines from 1 looks like the least interesting choice in the chapter, and it's one of the oldest arguments in programming, with a famous referee. Edsger Dijkstra was a Dutch computer scientist, one of the generation that founded the field, and for over forty years he mailed numbered notes to a few dozen colleagues. In 1982 he wrote one of them, EWD 831, titled "Why numbering should start at zero." His case goes through ranges first. Of the four ways to write "2 up to 12" with less-than signs, he picks the one that takes the bottom in and leaves the top out, $2 \le i < 13$: two ranges that meet end to end then share one number written once, and the length is just the top minus the bottom. Start a sequence of N things at zero

and it's $0 \le i < N$, and each thing's number is how many things came before it. He reports that programmers on the language Mesa, which allowed all four conventions, found the other three a steady source of clumsiness and mistakes, and he points out that FORTRAN starts at one. Does that settle it? It settles which convention is tidier, by an argument you can check. It doesn't settle which convention somebody else's work used, and that's the catch: the two-counter machine used by Forster, Larchey-Wendling and coauthors numbers its lines from 1, and so does this machine, because matching it line for line is worth more here than tidiness. Is zero the right place to start counting? Dijkstra's title says it is, and I can't tell you more than that. What his note carries is an argument about which way makes fewer mistakes. That part you can check for yourself, and it's the more useful thing to have.

Everything in that list is the *core*, the part the base moves. Outside the core sit the two pieces from the model: the ledger μ and the flag. The machine never reads either of them to decide what to do next. I'm not asking you to take that on trust: it's how the step is written, and it shows up as a theorem below.

The properties a fact can name come from a tiny fixed language with three words: *is zero*; *is even*; and *is at least n*, for any whole number n you like. Each one comes in two forms. There's the *checker*, a little program that computes yes or no: compare with 0, look at the last bit, compare with n. And there's the *meaning*, the plain arithmetic statement the checker is supposed to be checking: the number is 0; the number is two times something; n is less than or equal to the number. Why both? Because a checker is code, and code can be wrong in ways that look right, and "the checker says yes" is only worth anything once somebody has proved it says yes exactly when the meaning is true. That proof is one line per property, and it's the first block below.

Why can a machine check a claim at all? Because a claim with a precise meaning is something you *evaluate*: plug in the values and turn the crank. George Boole worked out in 1847 that you can do arithmetic on claims this way, treating "and" like multiplication and "or" like a kind of addition. Why multiplication? Try it at the table. Write 1 for true and 0 for false, and multiply: one times one is one, and anything times zero is zero. That's "and" to the letter. Then comes the stranger thing Boole noticed. Say "x and x" and you've said nothing new; it's just x. In numbers that's $x \times x = x$, and he made it a law of his system, and it's a law ordinary numbers mostly break. Which numbers equal their own square? Two times two is four, no. A half times a half is a quarter, no. Zero, yes. One, yes. That's the whole list. So when Boole wrote his bigger book in 1854, *The Laws of Thought*, he asked you to imagine an algebra whose letters take the values 0 and 1 and nothing else. Every checker on this machine answers in that algebra.

And the 1847 book has a backstory worth a detour, which I found in the Stanford Encyclopedia of Philosophy's entry on Boole. Boole was largely self-taught and spent

about fifteen years as a schoolmaster, and what pulled him back to logic, early in 1847, was a loud public quarrel between Augustus De Morgan and the Scottish philosopher Sir William Hamilton over who had first thought of a small technical idea in the old logic of syllogisms. Two well-known men fighting over credit, and a schoolmaster watching from the side picks up the subject and writes the little book this paragraph is about. Few people remember what the quarrel was over. Every checker in this chapter runs on what the bystander wrote.

Then push on it, because Boole's "or" isn't quite the one a checker uses. His plus only made sense for things that can't both hold at once, so "x or y" with both of them true had no meaning in his system. William Stanley Jevons, an English economist who also worked on logic, read "or" as "either or both," which gives "x or x is x," the same trick as before. He wrote to Boole in 1863 saying plus ought to work that way, and in 1864 he put it in a book called *Pure Logic*. Boole wouldn't have it, since it broke the tie to ordinary algebra his whole system stood on. The "or" in every programming language I know is the inclusive one Jevons argued for. Does that change anything about the checker? No; the crank turns the same way. It changes the picture: the "or" a checker uses is the one the man who invented the algebra turned down.

Hanging a checked claim on a point in a computation is an old move too. Tony Hoare, in 1969, built a whole way of reasoning about programs out of it: attach a claim to a *point* in a program ("whenever execution reaches here, this is true"), and reason forward from claim to claim. Robert Floyd had done it two years earlier on flowcharts. Their claims were checked on paper by the person writing the proof. This machine's version is a property of one counter at one edition, checked by a step while it runs.

And is two years earlier the bottom? Not even close, and the next step down is a paper this book already leaned on. In June 1949, at a Cambridge conference held for the new EDSAC computer, Alan Turing gave a talk three pages long, "Checking a large routine," the same note the counter chapter used for stopping (Section 8). Stopping was the second half of it. The first half is this move, twenty years before Hoare: the programmer should make "definite assertions which can be checked individually," and the correctness of the whole should follow from them. His example program computes a factorial using nothing but addition, and his picture for why you'd bother is one you can do on the back of an envelope. Add up a long column of four-digit numbers and hand someone only the total, and they have to redo the whole sum at one sitting, because the carries tie every column to the one before it. Hand them the total of each column as well, and the job falls apart into small checks they can do one at a time. Each column total is a claim hung on a point in the middle of the work, so the person checking never has to hold all of it at once. The paper got its due mostly later: in 1984 two computer scientists, F. L. Morris of

Syracuse University and C. B. Jones of Manchester University, who had worked on it together at Oxford, republished it, with its misprints corrected, as an early program proof by Alan Turing. Theirs is the text I read, and the column of numbers is Turing's own example in it. Does any of that change the machine? No. It changes the date on the idea, and it tells you something I like: the first person to ask how you'd check a program put the claims in the middle of it. He didn't save them for the end.

The machine has six instruction kinds.

Instruction	**What it does, when the trap is down**	**Cost**
INC *c*	Adds one to counter *c*, adds one to its version, goes to the next line.	0
DEC *c j*	If *c* is above 0: subtracts one, adds one to its version, jumps to line *j*. If *c* is 0: just goes to the next line.	0
HALT	Nothing. A program that reaches it stops.	0
CHECK *p c*	Runs the checker for *p* on *c*'s value. If it says yes and the table has fewer than sixteen facts, writes the fact (*p*, *c*, *c*'s current version) into the table and goes to the next line. Otherwise, trap.	1
COMMIT *p c*	Looks in the table for exactly the fact (*p*, *c*, *c*'s current version), field by field. If it's there, puts it in the channel and goes to the next line. Otherwise, trap.	1
CERTIFY	If the channel holds a fact, preserves that channel, goes to the next line, and the flag goes up. Otherwise, trap.	1

The first two are Minsky's machine, and HALT lets a program say it's done. It says it by doing nothing: HALT leaves the state where it is. If it put the trap up instead, it would land a live state on its trapped twin, a merge in the record's part, and the record-layer merge would charge it. The last three are the record's, and they read

as a chain: you can only certify with a commitment in the channel, you can only commit to a fact in the table, and the only way into the table is a check that passed. The chain has three links, and each one costs.

One of what? One unit of μ, the same unit as Section 2's toll. Why one? Why would ten be worse? Because one is the least each of these three acts can cost once a merge in the record's own part of the state is priced, as the end of this paragraph says, and Section 3 is where that pricing comes from. The argument for a whole finite machine doesn't reach this one: these counters have no top, and some of the base moves merge states at a price of zero. What does reach it is the record-layer merge named at the start of this chapter (Theorem 5.1, in Part I), which gives the toll once you've agreed to price a merge in the record's own part of the state. Agreeing to that is my choice. Once I have, one unit on each of the three acts isn't: COMMIT and CERTIFY merge when they pass, and all three merge when they fail, because a failure puts the trap up and changes nothing else, so the failed state lands where the same state with the trap already up lands. What I pick is one rather than more.

The whole numbers have a history of their own. In 1888 the German mathematician Richard Dedekind published a little book whose title asks what numbers are and what they're for, and in 1889 the Italian mathematician Giuseppe Peano published a booklet, written in Latin, that set out the axioms for the whole numbers in the form people still teach: there's a first number, and every number has a next one, and a few rules about how "next" behaves. I had the two of them down as working independently. I went looking, and the Stanford Encyclopedia's entry on Dedekind says Peano acknowledged Dedekind's priority, which is why the axioms often go by both names. (Peano's booklet is also where the sign $\in$ that you met in Section 7 comes from: he wrote it as a Greek epsilon, for the Latin *est*, "is.") Start from zero and take "the next one" three times, and you have three. Once prices are whole numbers, one is the smallest price that isn't nothing.

The zeros matter just as much: computing costs nothing on this ledger. Plus one, minus one and jumping around cost zero, with physical costs left to an implementation model. Only the three acts that make a claim cost, and each of them costs whether it passes or traps. Failing is not a discount.

The version rule. Some numbers show why versions exist. Start with $A = 4$, version 0. CHECK *is even* A passes, and the table gets the fact (*even*, A, 0). Now INC A: A becomes 5, and its version becomes 1. Now COMMIT *is even* A. The machine looks for (*even*, A, 1), the claim about the current edition. The table only has (*even*, A, 0). There's no match, so the machine traps. The old fact is still sitting in the table, still true about edition 0, and completely useless for committing to anything about edition 1. A fact is about one edition of one counter, and an edition never comes back, because versions only go up and never wrap. The version rule is the whole trick against stale evidence: you can't check a number, change it, and then

commit to the check. And the never-wrap part isn't a convenience. INC A, then DEC A back to the same line, puts the line and both counters exactly where they were, at no cost, and once record-layer merges are priced neither move may join two states that agree on those. If the record had only finitely many settings over one line and one pair of counters, enough round trips would bring the whole state back to where a CHECK left it, with A touched in between and its COMMIT still passing. Under those free-move, injectivity and continued-use requirements, the record fibre cannot be finite. This machine supplies fresh editions without a bound. A bounded implementation can instead stop before wraparound or change its invalidation and pricing rules; that changes the contract.

What the version rule stops is the time-of-check-to-time-of-use bug from Section 15, the file checked and then swapped, and the database answer to it is Kung and Robinson's: at commit time, check that nothing you read was written in the meantime, and back out if something was. The version rule is that check, done by the step, on one counter. It also shows where the guarantee ends. It works because one step does every write and every write bumps a version. A second writer that changed a counter without bumping its version, a second processor or a device writing memory on its own, would walk right around it, and none of the theorems below are about a machine with one of those.

Processor designers put a cousin of this check in silicon. Jensen, Hagensen and Broughton, at Lawrence Livermore in 1987, described a pair of instructions now called load-linked and store-conditional: the load reads a word and the processor starts watching it, and the store goes through only if nothing has written that word since; otherwise it fails and the program tries again. It's tempting to say CHECK is the load and COMMIT is the conditional store, and it's wrong in two places. COMMIT doesn't write the counter it checked, and the thing doing the watching, the fact, sits in a table the step keeps. No single processor holds it. The hardware version does cover the second writer, because the watching happens on the memory everyone shares, which is exactly the case the version rule leaves out.

The real kinship is older, and it's somewhere else. Picture the table and the counters shared by many processors, with CHECK, COMMIT and CERTIFY each done as one indivisible step. Maurice Herlihy (1991), a computer scientist who has spent his career on how many processors can share memory safely, ranked shared objects by how many processors each one can bring to agree on a single value with nobody ever waiting on anybody. Test-and-set, an instruction that writes a cell and reports what was there, brings exactly two. A sticky bit, the idea of Serge Plotkin (1989), a Stanford computer scientist working on distributed algorithms, is a cell the first writer jams for good, with the first writer's value, and everyone after can read it; it brings any number. I guessed that the shared object would sit at the top with the sticky bit, because a write that kills every fact about a counter, together with a

commitment other processors can see, looks like a way to build one. Ranking the object takes its operations, what each one returns, which steps are indivisible, how it starts and which processes run it, and then a proof that every processor decides, they all decide the same value, the value is one somebody proposed, and nobody waits. That proof goes through with every instruction one indivisible step and the whole state readable, and it puts the object at the top. The reason is the order of the fact table. Start it clean, with counter A at zero and the trap down. Each processor CHECKs a property that holds of A at zero, Zero to propose 0 and Even to propose 1, then reads the table and decides the bit of its oldest fact. Nobody touches A, so the first CHECK of the run passes. The table never loses an entry or reorders the ones it has, and a CHECK that finds it full trips the trap, after which nothing moves, so the first CHECK of the run fixes the oldest fact for good. Every processor that decides decides that bit, the bit is somebody's proposal, and each one decides after two of its own steps whatever the others do. The version rule and the commitment play no part. The trap alone doesn't do it: it records that something trapped, and it doesn't hand everyone the first writer's value.

Now take the version rule away, let failures change nothing, keep only the properties "at least n" so that a checked property only ever turns true, drop the decrement, and make the fact table (with no room limit), the commitments and the flag sets and latches that only grow. I guessed that then the object drops to the bottom rung, the level of plain memory, which is the shared-memory face of what the database researchers Joseph Hellerstein and Peter Alvaro (2020), at Berkeley and Santa Cruz, call CALM, where monotone means nobody has to coordinate. That guess is proved too, by Herlihy's own kind of argument. Let the updates return nothing and let the whole state be readable. If at every state, of any two updates, one leaves the state alone, or the two commute, or one overwrites the other, then no protocol brings two processors to agreement with nobody waiting. To see why, give one processor 0 and the other 1. At the start the answer isn't settled, since either one running alone has to decide its own bit. Nobody waits, so it can't stay unsettled forever, and there's a last unsettled state: whichever processor moves next settles it, one way if it's the first and the other way if it's the second. Look at those two moves. If one is a read, or an update that leaves the state alone, the other processor can't tell whether it happened, and running alone it decides the same bit either way. If the two commute, both orders end on the same state with both processors in the same place. If one overwrites the other, the overwriter running alone can't tell whether the other move happened first. Each case contradicts the two ways the answer was settled. The monotone object, with registers beside it, has that property. An update that fails changes nothing. Two updates that both succeed still succeed in either order, because an instruction that would succeed still succeeds after anything else has run, and adding to sets and to counters gives the same result in either order.

Two of the conditions do real work there. The table needs no room limit: a full table that turns away the latecomer is a first-writer-wins cell again, and that's back at the top. And the ledger: it changes neither result as long as a failed act in the variant isn't charged, or nobody reads the ledger. Charge a failed CHECK where everyone can read the total, and one processor INCs while the other CHECKs *at least 1*; the total and the table then tell each of them which act came first, and that's enough for two to agree. Neither idea is mine: the top is Plotkin's and the bottom is the monotone fact that field already knows.

Process-calculus people have a statement of the same split. Johannes Åman Pohjola and Joachim Parrow, computer scientists at Uppsala University (Parrow is one of the inventors of the pi-calculus, a mathematical model of processes that talk over channels), in 2016 proved that no model whose parallel composition is monotonic, where a neighbouring process can never stop a step, has a consensus process. One of their examples of a model that does is test-and-set. Test-and-set isn't a sticky bit: in Herlihy's ranking it brings two processors to agree, and a sticky bit brings any number.

Their setting is small enough to go through here whole. A model is any set of processes with a step relation and a way to compose two processes that doesn't care about order or grouping. Composition is monotonic when whatever a process can reach alone, it can still reach with anything set beside it. A consensus process is one where any number of copies, run together, can end up agreeing on either of two outcomes; once a run has one outcome it can never show the other; and an outcome, once reached, survives having more copies, run any way you like, set beside it. Their theorem goes through in a few lines: run half the copies to one outcome and the other half to the other. If neither half can block the other, the whole crowd can reach the pair of end states set side by side, and since each outcome survives the company, both outcomes hold there, which is forbidden. Then the machine goes into it. A process is a shared machine state with a crowd of threads, each proposing a bit. Composing two processes pools their threads, each thread keeping whatever it has already decided, and settles the shared state like this: if one side is still at the fresh start, the composite takes the other side's state; if both have already moved, the composite is jammed, a state where no thread can take a step. That composition doesn't care about order or grouping: a fresh start is neutral, and any two moved states jam whichever way round you put them. One thread proposing each bit is a consensus process in exactly their sense, for the outcomes "some thread decided 0" and "some thread decided 1": copies started fresh can reach either outcome and never both, because the first passing check fixes the oldest fact and every later thread reads the same one; and an outcome survives company, because pooled threads keep their decisions. So by their theorem the composition isn't monotonic, and the small machine sits on their non-monotonic side. That verdict is cheap, though:

a composition that jams any two moved states makes every shared-state machine non-monotonic, so the encoding puts it there, not the fact table. What does the work in the run, making every thread read the same outcome, is the order of the fact table, the same as above; a store that refuses a stale commit for good, as the version rule does, refuses in the same spirit. The shared-memory version of their split is the pair of results above: the small machine's object reaches agreement for any number of processors, and the monotone variant, with its table unbounded and no charge for failing that anyone can see, doesn't reach it even for two.

Security people know the pattern by a third name. A check that blesses a value, and a write that takes the blessing away, is close kin to dynamic taint tracking, where a label rides along with the data and a sanitizing step changes it; the labels go back to the computer-security researcher Dorothy Denning's 1976 lattice model of how information is allowed to flow. How much a monitor like that has to remember is a studied question too: Chamseddine Talhi, Nadia Tawbi and Mourad Debbabi (2008), three security researchers at universities in Quebec, worked out which rules a monitor can still enforce when its memory is bounded. The version rule is one small, specific member of that family, done by the step.

Notice what *isn't* a write. DEC on a counter that's already 0 doesn't change the counter, so it doesn't change the version. It's a test that happens to move the line number.

The fact table. Facts go in and never come out. Nothing erases them and nothing overwrites them. When the table is full, a seventeenth CHECK traps rather than pushing an old fact out to make room. That choice has a downside, and I'll set it out in the subsection on what this machine doesn't do.

Traps. Any failure puts the trap up and changes nothing else in the core; the ledger still takes the instruction's cost. Once it's up, every instruction leaves the core alone. A stored program stops at a trap. But I also prove things about *any* list of instructions, programs or otherwise, because an adversary doesn't have to run my programs; they can feed the machine whatever they want. For those lists, an instruction after a trap is still charged its cost, and still changes nothing. That trap keeps nothing of what tripped it, and the record-layer merge charges it for that. Merely counting refusals does not remove the merge. A live state with count zero and its already trapped twin with count zero can both become trapped with count one. Keeping the entire previous state in a history would make each extended move injective, but that is a different, unbounded state space. The counter alone is not that history.

The exact version comes next.

Definition 1.1 (Properties, checker, meaning)**.** A *property* is one of: Zero, Even, or Ge(n) for a whole number n. The *checker* eval(p, v) computes yes or no: $v = 0$ by equality, evenness by the parity function, $n \leq v$ by comparison. The *meaning* holds(p, v) is the arithmetic statement: $v = 0$; there is an m with $v = 2m$; $n \leq v$.

□ **Lemma 1.2** (The checker says what it means)**.** For every property p and every whole number v, eval$(p, v) =$ yes if and only if holds(p, v).

Proof. Ask whether the number is zero: the equality test says yes at zero and no at every other number. Ask whether it is at least n: the comparison says yes precisely when n is no larger than it. Those two tests already answer the arithmetic questions they name.

For evenness, the checker says yes at zero, no at one, and adding two leaves its answer unchanged. Take any whole number and remove pairs until fewer than two remain. If none remain, the number was two times the number of pairs removed, and the checker gives zero's answer, yes. If one remains, it was two times that many pairs plus one, and the checker gives one's answer, no. Thus yes means exactly ■ $v = 2m$ for some whole number m. These are all three properties in the language.

Definition 1.2 (Core, state, claim, start)**.** A *core* k holds two counters A, B and their versions (whole numbers), a line number, a fact table (a list of facts, a fact being a triple of a property, a counter and a version), a channel (empty or one fact), and a trap bit. Write val(k, c) and ver(k, c) for counter c's value and version. A *state* s is a core, written core(s), with a ledger $\mu(s)$ and a flag cert(s). The *claim* p of c in k is the fact $(p, c, \text{ver}(k, c))$. The table's room is 16. The start start(a, b) has counters a and b, versions 0, line 1, an empty table, an empty channel, the trap down, $\mu = 0$ and the flag down. A state is *clean* when its table and channel are empty and its flag is down.

Definition 1.3 (Instructions, costs, the step)**.** The instructions are INC c, DEC c j, HALT, CHECK p c, COMMIT p c and CERTIFY, with effects on a core with the trap down as in the table above. Passing any check, commitment or certification requires the trap to be down. With that condition, CHECK passes when eval$(p, \text{val}(k, c)) =$ yes and the table holds fewer than 16 facts, COMMIT passes when the claim p of c is in the table, and CERTIFY passes when the channel holds a fact; a failing one sets the trap and changes nothing else. On a core with the trap up, every instruction

changes nothing. Costs: CHECK, COMMIT and CERTIFY cost 1, the rest 0. One step on a state is

$$\mathrm{exec}(s,i) = \big(\,\mathrm{cexec}(\mathrm{core}(s),i),\ \ \mu(s)+\mathrm{cost}(i),\ \ \mathrm{cert}(s)\vee\mathrm{fires}(\mathrm{core}(s),i)\,\big),$$

where cexec is the core's step and $\mathrm{fires}(k,i)$ is yes exactly when i is CERTIFY and passes at k. For any list t of instructions (a *trace*), $\mathrm{run}(t,s)$ applies them in order and $\mathrm{cost}(t)$ adds their costs. A *program* is a list of instructions numbered from 1; a program run fetches the instruction at the line number and executes it, and stops when the trap is up, the line holds HALT, or the line holds nothing.

Read the step line slowly, because the rest of the chapter leans on its shape. The core moves by its own rule and nothing else. The ledger adds the cost. The flag becomes "it was up, or this is a CERTIFY that passed." The core's move doesn't mention μ or the flag anywhere. The base is blind to the record by construction.

The shortest worked run there is uses the program CHECK *is zero* A; COMMIT *is zero* A; CERTIFY. Start at $\mathrm{start}(0,b)$, for any b. Line 1: A is 0, the checker says yes, the table is empty so there's room, and the fact (*zero*, A, 0) goes in. $\mu = 1$. Line 2: the claim about A's current edition is (*zero*, A, 0), and it's right there, so it goes into the channel, and $\mu = 2$. Line 3: the channel holds a fact, the flag goes up. $\mu = 3$. Line 4 has no instruction, so the program stops. The run ends with three paid acts, in order, a flag that's up, and a channel that names the last earned commitment, which here is (*zero*, A, 0).

Now start the same program at $\mathrm{start}(5,b)$. Line 1: A is 5, the checker says no, the trap goes up, $\mu = 1$, and the program stops right there. The flag is down, and you can run it as long as you like: it stays down. That second run is the point of the first. The same three instructions certify from some starts and refuse from others, because the check could have failed, and from most starts it does. A check like that has a no pile, which is the only kind Section 15 counts as establishing anything.

Could I have picked an even dumber claim? I could: CHECK *at least 0* A passes on every number there is, so CHECK, COMMIT, CERTIFY on it raises the flag for 3 from every start. The step doesn't ask whether the question was worth asking, and it shouldn't. Worth is a count over the rivals a run narrows (Section 10), and a step that sees one state has no list of rivals to count. What the step checks is order, aboutness and that the checker tells the truth. What a check that can't fail is worth, the entitlement count says exactly: zero bits, for a cost of three. The price is for standing behind the claim. What the claim bought is a separate number, one the step doesn't compute, and I keep the two apart. It's on the list at the end of the chapter.

Others walked into the same line from communication. Three theoretical computer scientists, Oded Goldreich, Brendan Juba and Madhu Sudan (2012), built a

theory of communication with a partner you may not understand, in which getting a message across counts only as a way to reach a goal, and the central notion is *sensing*: being able to tell, from what you see, whether you're making progress. They ask two things of it. It has to be *safe*: with any partner in the class, when there's no progress it keeps saying so, so it can't report a lasting success that didn't happen. And it has to be *viable*: with a partner who could help if you spoke its language, the right way of speaking gets a yes from it within a known time, and keeps getting one. Either half alone is cheap. A sensor that always says no is safe, and one that always says yes is viable. Their main result, as I read it, is that the two together, with the ways of speaking listed one after another, are enough for one strategy that reaches the goal with every such partner. Their theorem, in their setting, is a check that can fail and never lies, standing in front of a verdict you can't take back, and it's the same shape.

1.4 It computes everything two counters can

First the flat part, because the example is meant to run every program, and that has to be shown. Put the ordinary two-counter program beside this machine's program. Replace each "add one" by this machine's INC, and each "subtract if possible and jump" by its DEC, keeping the named counter and jump destination. That is all the translation does.

Now cover the facts, versions, channel, flag and ledger with a sheet. Leave the two counters and line number visible. I want to show that the visible part follows the original program exactly. The hidden versions may change, but these instructions cannot raise the trap, so the sheet never hides a new refusal that would stop the comparison. Here are the two programs and the window written precisely.

Definition 1.4 (Two-counter program, compile, window)**.** A *two-counter program* M is a list of instructions, each either MINC c or MDEC $c\ j$. A configuration is a triple (line, a, b). One step of M: if the line holds no instruction, M has *stopped*; MINC adds one to c and goes to the next line; MDEC subtracts one from c and goes to line j if c is above 0, and goes to the next line if c is 0. $\mathrm{mrun}(n, M, x)$ is n steps from x, staying put once stopped. The *compile* of M replaces each MINC c by INC c and each MDEC $c\ j$ by DEC $c\ j$. The *window* of a core k is the triple (line number, $\mathrm{val}(k, A)$, $\mathrm{val}(k, B)$).

Theorem 1.3 (Step for step)**.** For every two-counter program M and every core k with the trap down: M has stopped at window(k) if and only if the program compile(M) has halted at k; and if M steps from window(k) to y, then one program step of compile(M) from k lands on a core with window y and the trap still down. So for every n, n program steps of compile(M) from k land on a core whose window is mrun$(n, M,$ window$(k))$, with the trap down. □

Proof. Look up the instruction at the line. If there is none, both sides stop. If it's MINC c, the compiled INC c adds one to c and goes to the next line, which is exactly the window's move (the version also moves, but the window doesn't show versions). If it's MDEC c j, split on whether c is 0, and both sides do the same thing in each case. Neither instruction can trap. After that move the windows agree again and the trap is still down. Those are exactly the conditions I began with, so the next instruction can be compared in the same way. Repeat until the chosen n steps have been counted; if either side stops earlier, both stay put for the remaining comparisons. ■

Theorem 1.4 (Halting matches)**.** For every two-counter program M and whole numbers a, b: M stops after some number of steps from $(1, a, b)$ if and only if the program compile(M) halts after some number of program steps from start(a, b). □

Proof. The start has window $(1, a, b)$ and its trap is down. Apply Theorem 1.3 after any number n of steps: the two configurations still have the same window, the trap is still down, and one is stopped exactly when the other is halted. The same n witnesses termination in either direction. ■

Now comes the part that isn't mine. Two-counter universality is Minsky's, and the constructions of Forster, Larchey-Wendling and their coauthors connect it to several other models. The counter convention matters: DEC at zero goes on to the next instruction; otherwise it takes one off and jumps, the convention used in Andrej Dudenhefner's certified decision procedures for two-counter machines (2022). Their route runs from a tape to counters and packs finitely many counters into two. The remaining step here is to match that two-counter machine to this machine's INC/DEC fragment, instruction for instruction.

I want to be exact about what "undecidable" means in the theorem below, because it's stated as a connection between two problems. Suppose you had a procedure that answered the halting question for this machine. Then you could write a program

that lists, one by one, every single-tape Turing machine that *doesn't* halt. No such list can exist: the ones that *do* halt can already be listed (just run them all, a step at a time, and call out each one as it stops), and with both lists running side by side you'd have a halting decider. So the theorem says: a decider here would give you that impossible list.

□ **Theorem 1.5** (Halting is undecidable for this machine)**.** The two-counter machine with the instruction convention just stated agrees with the INC/DEC fragment: its step, its stop, and its termination match those of the compiled program, so its halting problem at (M, a, b) has a positive answer if and only if compile(M) halts from start(a, b). Consequently, the question "does program P halt from start(a, b)?" is undecidable in the following reduction sense: a decider for it would make the complement of single-tape binary Turing machine halting enumerable.

Proof. The agreement is a case split on the counter-machine instruction at the current line, and then induction along the run, ending in Theorem 1.4. Composing a proposed decider for the compiled program with the compile map would decide two-counter halting. The tape-to-counter reduction would then decide single-tape ■ binary halting and so list its complement, as claimed.

What that buys is that the base of this machine is a universal machine in Minsky's sense, and that no algorithm can predict in general what it'll do. What it doesn't buy is a fast machine, or one where your input arrives conveniently. The counters start holding a and b, given directly; there's no encoding layer in this construction. Minsky's prime-power packing is how you'd get from "any Turing machine" to "two counters," and I use that construction as it stands.

Is "given directly" just a convenience, or does it bite? It bites, and somebody found exactly where. In 1972 Rich Schroeppel, then at the MIT AI Lab, wrote a ten-page memo titled "A Two Counter Machine Cannot Calculate 2^N." He spells out the convention first: a counter machine computing a function starts with n in one counter and zero in the others. Then he gives a proof that a two-counter machine started that way can't compute 2^n. That's his theorem, and I don't reprove it. A cousin holds on this machine: no program multiplies every input by one large constant fixed by the program's length. So the packing Minsky needs is precisely the thing two plain counters can't do for themselves. Hand the small machine its inputs straight and there are functions of them it can't compute; hand it the inputs already packed and it computes anything a Turing machine can. That's no contradiction with the undecidability above, which ranges over every program and every starting pair, packed ones included. The base hides nothing; it just needs its input pre-chewed, and the chewing is somebody else's.

1.5 It's a Thiele machine, and the flag is a latch

Now it's the record's turn. There are three facts to state first, and all of them read straight off the step line.

Theorem 1.6 (Ledger, toll, permanence)**.** For every state s and instruction i, $\mu(\text{exec}(s,i)) = \mu(s) + \text{cost}(i)$, and for every trace t, $\mu(\text{run}(t,s)) = \mu(s) + \text{cost}(t)$. If $\text{cert}(s)$ is down and $\text{cert}(\text{exec}(s,i))$ is up, then i is CERTIFY, it passes at $\text{core}(s)$, and $\text{cost}(i) \geq 1$. If $\text{cert}(s)$ is up, so is $\text{cert}(\text{exec}(s,i))$. □

Proof. Each instruction adds its listed charge to the ledger. Start at $\mu(s)$: after the first instruction its charge has been added, after the second both charges have been added, and every further instruction adds its own. An empty list adds nothing. Thus any finite trace leaves exactly the starting ledger plus the sum of the charges.

The flag has two ways to be up afterwards: it was already up, or this instruction successfully certified. If it was down before and up afterwards, only the second way is available. The instruction must be CERTIFY, its check of the channel must pass, and its listed charge is one. If the flag was already up, the first way keeps it up regardless of the next instruction. ■

The middle clause of those three is Section 2's toll, for this machine. The last is permanence. And the run of Theorem 1.20 raises the flag from a clean start. Put together, they mean this machine *is* one of the things Section 2 was about, and Section 2's universal floor applies to it without my lifting a finger.

Theorem 1.7 (The small machine is a Thiele machine)**.** The states, the instructions, the step exec, the cost and the flag, with the states $\text{start}(a,b)$ as the named starts, form a Thiele machine (Definition 2.2). So the universal floor (Observation 2.2) holds here: if $\text{cert}(s_0)$ is down and $\text{cert}(\text{run}(t,s_0))$ is up, then $\text{cost}(t) \geq 1$. □

Proof. Take the state and instruction sets of Definitions 1.2 and 1.3, their step and cost, the flag as the reading, and the states $\text{start}(a,b)$ as the named starts. These provide the five components of Definition 2.2. Theorem 1.6 gives the toll and permanence, and Theorem 1.20 gives a run from $\text{start}(0,b)$ that ends with the flag up. Applying Observation 2.2 gives the claimed bound. ■

Section 12 comes next. Section 12 said: take any base, and any record that answers only to the base and never comes back down, and the record is a latch. This machine has a base: a program together with a core, stepping by the program step. And it has a record, the flag, with the ledger riding along. So the question is

whether it's a *base-driven* extension in Section 12's five-rule sense (Definition 12.2), and if it is, Section 12 does the rest.

□ **Theorem 1.8** (The flag is a latch of the base)**.** Let the base have states (program, core), stepping by the program step, starting at start(a, b)'s core for any a, b, and halting when the program has halted. Map each (program, state) to (program, core). Then the machine, run as a closed machine on (program, state), is a base-driven record over that base, and so its flag factors as the latch (Observation 12.1) of the event "the next instruction is a CERTIFY that passes."

Proof. First the cover (Definition 12.1): the map sends starts onto starts, it commutes with the step because the core's move never reads μ or the flag, and whether the program has halted is read off the core. Then the rules of Definition 12.2 one at a time. Driven by the computation: the next flag is the old flag or "the next instruction is a passing CERTIFY," which is a function of the base state and the old flag (that's the step line). Ledger: μ only adds. Toll: Theorem 1.6. Permanence: Theorem 1.6. A reachable write: the three-line program of the worked run raises the flag from a start. Section 12's theorem then gives the factorization. ■

So far there's nothing new. The toll, the floor and the latch are things Parts I and II already promised about any machine of this kind. The next subsection is the part neither of them could give.

The lamp and the claims it does not name

The lamp is on. Which claim turned it on? Did another claim get certified afterwards? Look at the lamp as long as you like. It still gives you one answer.

Put a file beside it. Every time CERTIFY succeeds, copy the fact from its channel into the file: the predicate, the counter it concerns, and that counter's edition. Call the set of filed facts R. Publishing the same fact again leaves the same set. A later write can change the counters while the old receipt stays filed under its old edition.

Run this record alongside the native state. The native instructions keep doing exactly what they did before. Forget the file and you recover their run. Open the file and you can ask which claims were actually published.

□ **Theorem 1.9** (The native lamp is the archive's Boolean reading)**.** From a clean start with an empty added archive, after every finite native instruction word,

$$\mathrm{cert}(s) = \mathrm{true} \quad \Longleftrightarrow \quad R \neq \varnothing.$$

Membership in R means exactly that this fact was emitted by a successful CERTIFY earlier in that word. Already recorded facts persist, and zero-cost instructions cannot add a fact.

Proof. Start with the empty archive and the flag down. Inspect the next instruction. A successful CERTIFY inserts its channel fact into R and raises the flag. Whether or not that fact was already present, the resulting set is nonempty and the flag is up. Every other case leaves both the archive and the flag alone, including failed or trapped instructions. This includes every horizontal move. It follows, one instruction at a time, that a fact belongs to R exactly when some earlier successful CERTIFY published it, that no later move removes it, and that the flag is up exactly when R is nonempty. Forgetting R leaves the native updates untouched, so it also recovers the native run. ∎

The lamp tells you whether that file is empty. The file tells you what is in it. Below, I will make 65,535 different nonempty files leave the entire native machine in one identical state. Before that, follow how a fact earns its place: CHECK establishes it in the table, COMMIT selects it, and a successful CERTIFY publishes it. Each step has a job.

1.6 Commitments are earned

The shape comes first, before the words. Pick any run from a clean start, any list of instructions at all, chosen by somebody who'd love to cheat. Look at the moment a COMMIT would pass. Then, somewhere earlier in that run, there's a CHECK of exactly the same claim that passed and paid, at the same edition of the counter, and between that CHECK and now, nothing touched the counter. That holds on every list of instructions anybody can write, not "probably" and not only "on the programs I wrote."

To say "nothing touched it" exactly takes one word.

Definition 1.5 (Untouched)**.** A list of instructions *mid*, run from a state s, leaves counter c *untouched* when, for every way of splitting *mid* as t_1, then i, then t_2, executing i at the core reached by running t_1 from s leaves both $\mathrm{ver}(\cdot, c)$ and $\mathrm{val}(\cdot, c)$ as they were.

Theorem 1.10 (A commitment was checked first)**.** Let s_0 be clean and *pre* any trace after which COMMIT p c would pass. Then *pre* splits as pre_1, then CHECK p c, then *mid*, where: the CHECK passes at the core reached by pre_1; CHECK p c and COMMIT □

p c each cost 1; c's version after pre_1 equals its version after pre; and mid, run from just after the CHECK, leaves c untouched.

Proof. COMMIT passes, so the claim $(p, c, \mathrm{ver}(c))$ for the current version is in the table. By the no-forging invariant (Theorem 1.14 below), that fact was written by a passing CHECK p c at some point pre_1, with the version c had then; since it's the current version too, the two versions are equal. Versions never go down, and any instruction that changes a counter's value also raises its version, so an instruction in mid that touched c would have left c's version higher at the end than at the CHECK. It isn't. So nothing in mid touched c. ■

□ **Theorem 1.11** (A raised flag was checked, then committed, then certified)**.** Let s_0 be clean and t any trace with $\mathrm{cert}(\mathrm{run}(t, s_0))$ up. Then for some p and c, t is

$$pre_1,\ \text{CHECK}\ p\ c,\ mid_1,\ \text{COMMIT}\ p\ c,\ mid_2,\ \text{CERTIFY},\ post,$$

where the CHECK passes at its state, the COMMIT passes at its state, the CERTIFY passes at its state, c has the same version at the CHECK as at the COMMIT, and mid_1 leaves c untouched. And when the CERTIFY runs, the channel holds exactly the claim that COMMIT made: the fact (p, c, c's version at the CHECK). CERTIFY takes no operand; this is what ties it to the claim.

Proof. The flag starts down, so some first instruction raised it, and by Theorem 1.6 it's a passing CERTIFY. A CERTIFY passes only with a fact in the channel, the channel starts empty, and only a passing COMMIT puts a fact there, so some earlier COMMIT p c passed. Take the last one before the CERTIFY: the fact in the channel is the one it wrote. Theorem 1.10 at that COMMIT gives the CHECK, the equal version, and the untouched stretch, and the equal version makes the committed fact (p, c, c's version at the CHECK). ■

The failure side comes next, so you can see the refusals are clean, with nothing left half-done:

□ **Theorem 1.12** (Unearned steps trap and change no claim)**.** If COMMIT p c would not pass at a core k, executing it puts the trap up and leaves the channel and the table as they were. If CERTIFY would not pass at a state s, executing it puts the trap up and leaves the flag as it was.

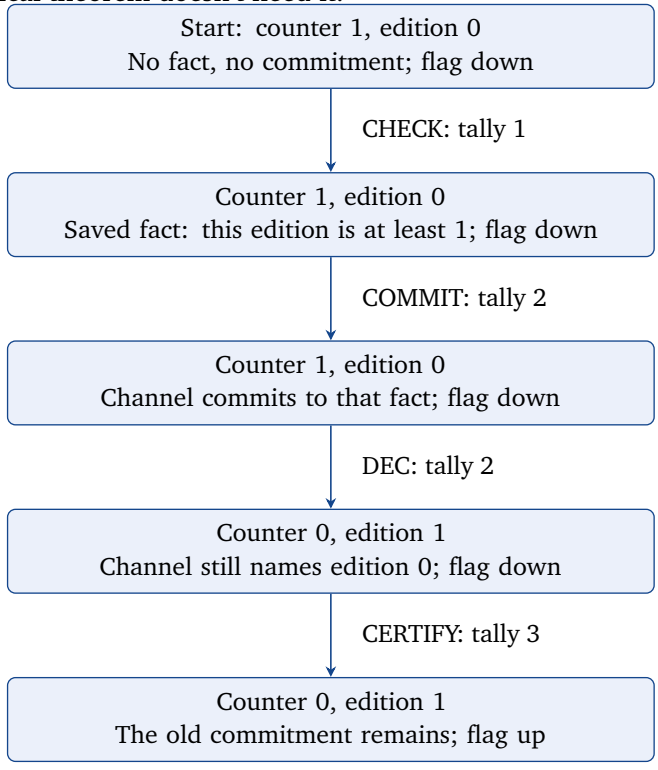
Proof. If the trap is already up, the core is unchanged and its trap remains up. Otherwise, the failed COMMIT takes the failure branch of the instruction table: set the trap, leave every other core field alone. A failed CERTIFY takes the same branch, and its fires value is false. The flag update is then its old value or false, which leaves it unchanged. The instruction's charge still applies to the ledger. ∎

If that rings a bell, it should. It's Milner's move from Section 15: in LCF a theorem is a value that only the rules of the logic can build, so nobody can just print one (Milner and two colleagues on his Edinburgh project, Michael Gordon and Christopher Wadsworth, wrote the system up in 1979, and Gordon, who went on to build the HOL prover at Cambridge, told its history in 2000). Here the thing nobody can print is one bit, and what guards it is one step rule. Milner's guard was a type checker. Same architecture, much lower down.

A provenance theorem is also an itemised receipt, and the itemised receipt has a history that's exactly this gap. James Ritty ran a saloon in Dayton, Ohio, and believed some of his staff were pocketing money. On a steamboat trip to Europe in 1878 he watched a mechanism that counted the turns of the ship's propeller, and wondered whether something like it could count the money instead. In 1879 he and his brother John patented a "Cash Register and Indicator," and they sold a version under the wonderful name Ritty's Incorruptible Cashier. The first model was this book's counter (Section 8) with brass on it: press a key, the amount goes onto a dial showing the day's running total. There was no drawer and no paper. At closing time you hold the dial up against the cash. And that's all a total can do. It tells you *that* the money is short. It can't tell you which sale went missing or who rang it up. The paper came later. The patents passed to other owners, first a Cincinnati salesman named Jacob Eckert, and in 1884 to John H. Patterson, who renamed the business the National Cash Register Company, and the story told about that company is that it added a paper roll: one copy kept for the shop's own books, one handed to the customer. That last part I only have from histories of the company and of the machine, so hold it the way you'd hold any company's story about itself. The order is right either way: first a total that can tell you something's wrong, then a list that tells you where. The ledger μ is the dial. Theorem 1.11 is the list: which CHECK, of which claim, about which edition, before which COMMIT. The theorem supplies that list about the run. The machine doesn't keep it as data anyone can read back from the end state, and nothing in the theorem gives a procedure for rebuilding it from there. A fact in the table holds a property, a counter and a version number. That's neither the counter's value back then nor a log of what happened in between.

Read Theorem 1.11 once more for what it *doesn't* say. Between the CHECK and the COMMIT, the counter is untouched. Between the COMMIT and the CERTIFY, it isn't promised anything. You can check A is even, commit to it, add one to A,

and then certify. The flag goes up, and what it stands on is a commitment about the edition that was current when you committed. The certificate is historical: it says "this was checked and committed about this edition," and it says nothing about whether that's still true. I think that's the right meaning for a certificate (a birth certificate doesn't expire when the baby grows), but it's a meaning, and you should know which one you're getting. So a certificate doesn't say the claim holds at the CERTIFY, and it doesn't say the claim holds at the end of the run. A CERTIFY that rechecked the version would give a different guarantee. That's a possible extension, and the historical theorem doesn't need it.



The checked claim was true of edition zero. The raised flag does not say that the current counter is still at least one.

1.7 What a passing check means

Provenance says a CHECK happened. It doesn't yet say the CHECK was *right*, and that's a separate question with a separate proof. A machine could have a perfectly ordered chain of check, commit, certify, all standing on a checker that says yes to everything. So what does a fact in the table actually tell you about the counter?

Theorem 1.13 (Checker soundness)**.** Let s_0 be clean and t any trace, and let k be the core of $\text{run}(t, s_0)$. For every fact (p, c, v) in k's table with $v = \text{ver}(k, c)$, the property holds of the counter's current value: $\text{holds}(p, \text{val}(k, c))$. In particular, if COMMIT p c would pass at k, then $\text{holds}(p, \text{val}(k, c))$. □

Proof. Carry one sentence through every instruction: every fact in the table has a version no higher than its counter's, and if its version equals its counter's, its property holds of its counter's value. A clean start has an empty table, so the sentence holds there. A passing CHECK adds a fact at the current version, and the checker said yes, so by Lemma 1.2 the property holds. A write raises one counter's version, so every old fact about that counter falls strictly behind and the second half no longer applies to it. Nothing else changes a value or a version. The sentence holds after every instruction, so it holds at the end. The part about COMMIT is the case where the claim is in the table at the current version. ■

So, on a run from a clean start, a live fact means exactly that one of three arithmetic properties holds of one counter's current value. It doesn't mean "the program is correct," and it doesn't mean "the number is interesting"; it means the property holds of the value, now. That's all "checked" means on this machine, and it's enough to be true. Three events stay separate here. The property holding of the value is one. A CHECK passing is another: a CHECK of a property that holds still doesn't pass when the table is full or the trap is already up. A certification is a third, and it needs a COMMIT and a CERTIFY after the CHECK. So a CHECK that didn't pass says its property is false only when the trap was down and the table had room before it.

So who checked the checker? Nothing on the record did. From a clean start, the record carries the run's half of the story: a CHECK, then a COMMIT of the same claim about the same edition, then a CERTIFY. The other half, that the checker says yes exactly when the property holds, isn't in any run. It's Lemma 1.2, a proof about the little program the step calls, priced by nothing on the ledger. So a claim established here stands on two legs: a fact about this run, carried by the record, and a fact about the checker, carried by a proof that whoever reads the record has to accept. That's the Münchhausen trilemma from Section 15 taking its third horn on purpose: the chain of reasons stops at a proof, named and on the page, where you can decide whether you'd have stopped there too. And the entitlement it gives is the reader's. The machine doesn't know anything. It keeps the order.

The question in that paragraph is older than any checker, and its most famous wording isn't about what you'd guess. "Who will guard the guardians?" gets quoted about governments and police and the people checking mathematical proofs.

Where's it from? Plato's *Republic* has the worry, sort of: in Book III (at 403e), Glaucon, Plato's older brother and one of the people Socrates is talking with, says it would be absurd for a guardian to need a guardian, which is close in words and opposite in mood, a man calling the question silly. The Latin, *quis custodiet ipsos custodes*, is from the Roman satirist Juvenal's sixth satire, and it's a joke about a husband. His friends tell him to bolt his wife in and set guards on her; the line asks who'll watch the guards, since she'll start with them. And it gets better. In 1899 an Oxford undergraduate, E. O. Winstedt, found extra lines of that satire in a medieval manuscript in the Bodleian Library, lines the other copies lack, partly overlapping the familiar ones, and the famous question sits in that tangle. Scholars have argued ever since about whether those lines are Juvenal's at all. So the most quoted sentence about trusting the checker comes with a doubtful source of its own. Does that change anything here? It doesn't. The answer used here is simple: you never get rid of the last guard, you make the last guard small enough to watch yourself. Lemma 1.2 is one line per property. Euclid had the instinct: five postulates you can hold in your head, and everything else meant to follow from them.

That split, a fixed checker proved sound from outside and a process that only gets to act when the checker says yes, has a name in the writing on self-improving programs. The AI researcher Jürgen Schmidhuber, at the IDSIA lab in Lugano, described a Gödel machine (2003) that rewrites its own code only after its built-in proof searcher has proved the rewrite useful, and Benja Fallenstein and Nate Soares (2015), researchers at the Machine Intelligence Research Institute in Berkeley, survey agents that reason about their successors through formal proofs a fixed checker accepts. Same architecture, aimed at a different question: they want to know when a machine may trust a smarter version of itself, and this machine only wants to know whether a flag was earned. Keeping the checker's proof outside the machine is my design choice. The nearest theorem, in Section 14, is narrower: no procedure whose flip the substrate can represent decides, for every program, a property that depends only on behaviour and holds of one program and fails of another. It doesn't say a machine can't check a proof about a checker, or verify a restricted part of itself.

Soundness rules out a lying checker. It doesn't rule out a pointless question: *at least 0* has a perfectly sound checker and an empty no pile, as the worked run showed.

Program checking has a theory of its own, and its founding paper asks for a checker that could have said no. The computer scientists Manuel Blum and Sampath Kannan (1989; journal version 1995) defined a checker for a computational problem: given a program and an input, it may run the program as it likes, and it must say "correct" with high probability when the program is right everywhere and "buggy" with high probability when its answer on that input is wrong. A checker that always says "correct" isn't one. Here the checker is fixed, deterministic and proved sound

once, and the part of their demand that survives is the fourth thing certified asks for: some claim whose check can fail.

1.8 Nothing gets forged

The proof of Theorem 1.10 leaned on an invariant I promised to show you, and this subsection shows it. The fear it answers: could some clever sequence of instructions get a fact into the table without a CHECK writing it? Could a DEC with a funny jump target do it, or a COMMIT after a trap, or something I didn't think of? I answer it one instruction at a time, which is the only way I trust myself to answer it.

Definition 1.6 (Earned fact; no forgery)**.** After a trace t from s_0, a fact $f = (p, c, v)$ is *earned* when t is *pre*, then CHECK p c, then *mid*, where the CHECK passes at the core reached by *pre*, f is exactly the claim p of c at that core, and, if f is live at the end (v equals c's version after t), *mid* leaves c untouched. The trace has *no forgery* when every fact in the table after it is earned.

Theorem 1.14 (No forging)**.** For every core k, instruction i and fact f in the table after executing i at k: either f was already in k's table, or i is CHECK of f's property and counter, it passes at k, and f is the claim it writes. Consequently, if a trace has no forgery, so does that trace followed by any one instruction; and from every clean start, every trace has no forgery. □

Proof. Inspect who can write to the table. INC and DEC change counters, versions and the line; they leave the table alone. HALT leaves it alone. COMMIT reads it and may write the channel, but cannot write the table. CERTIFY reads the channel and cannot write the table either. CHECK is the only remaining instruction. On failure it adds nothing; on success it adds exactly the property, counter and present edition it just checked. A trapped instruction changes no core field. That exhausts the ways a fact could appear after one instruction.

Now suppose the table's old entries already have their promised histories. A new entry gets its history from the passing CHECK just performed. For an old entry, that earlier CHECK still happened. If the entry is live after the new instruction, its saved edition equals the counter's edition now. The counter's edition never goes down. It therefore could not have risen away from the saved edition and then returned. In particular, it was already live before this instruction and the instruction did not change the edition or the value. Its untouched stretch extends by this one step. If the entry is stale, only its earlier passing CHECK is required, and that history remains.

At a clean start there are no entries to forge. The argument preserves that promise after the first instruction, after the next, and after each further instruction in any finite trace. ■

1.9 The window can't see the record

Section 9 was about windows: a view of a state that shows some of it, and the theorem that a window which shows the same before-and-after pair for a raise and a non-raise cannot price both exactly (Observation 9.2). The two runs below are the smallest pair I could build where the window is Minsky's view of the machine (line number and the two counters) and the record is everything else.

Run two traces from start$(0,0)$. The *earned* run is CHECK *is zero* A; COMMIT *is zero* A; CERTIFY. The *idle* run is DEC A 0, three times. In the idle run, A is 0 every time, so each DEC just goes to the next line and writes nothing. Both runs end on line 4, with $A = 0$ and $B = 0$, both versions 0. Through the window they're identical. One of them has paid 3, holds a fact, could commit again, and has its flag up. The other has paid nothing, holds nothing, can't commit, and its flag is down. They're like two photographs that show the same picture and are of different people.

□ **Theorem 1.15** (Same window, different record)**.** The earned run and the idle run from start$(0,0)$ end at cores with the same window and the same versions of A and B, and with different ledgers, different flags, different tables, and different answers to "would COMMIT *is zero* A pass."

Proof. In the earned run, CHECK adds $(\text{Zero}, A, 0)$, COMMIT puts that fact in the channel, and CERTIFY raises the flag. The ledger is $1 + 1 + 1 = 3$. No counter is written. In the idle run each DEC sees zero, so it advances the line without writing a counter or raising a version. Its three costs are zero and it adds no fact or commitment. So both final windows are $(4, 0, 0)$ with versions $(0, 0)$. The earned table contains the current zero claim and the idle table is empty, giving the stated difference in whether another COMMIT would pass, as well as in the ledger and flag. ■

□ **Theorem 1.16** (No window oracle)**.** There is no function g from windows to whole numbers with $g(\text{window}) = \mu$ at the end of every trace from start$(0,0)$. Likewise there is no function from windows to yes or no that gives the flag at the end of every such trace, and none that gives whether COMMIT *is zero* A would pass.

Proof. Such a g would have to give both 3 and 0 on the one window the two runs of Theorem 1.15 share. The same for the flag and the commit right. ∎

The record lives off the window, and that's Section 9's shape, made of pebbles. Anybody who only watches the counters and the line number, which is everything Minsky's machine *is*, can't tell you what was paid, what was committed, or whether the flag is up, and the reason isn't that they looked carelessly; it's that none of it is there to see.

Two things the same in everything you look at and still two things: philosophy has an old fight about exactly that, and it's worth a detour, because it tells you which side of the fight these theorems stand on. Leibniz said that for real things it can't happen. There are no two individuals indiscernible from each other, he wrote in 1716 to Samuel Clarke, the English philosopher and theologian who was taking Newton's side against him in an exchange of letters, and he told a story to go with it. A clever gentleman of his acquaintance, talking with him in front of the Electress Sophia of Hanover, Leibniz's friend and patron, in the garden at Herrenhausen she had helped lay out, thought he could find two leaves perfectly alike. She dared him to. He "ran all over the Garden a long time" looking, and it was "to no purpose." You can run his search tonight with a handful of anything, leaves or coins or grains of rice: look hard enough and every pair differs somewhere. Then in 1952 Max Black, a Russian-born philosopher teaching at Cornell, pushed back with a universe you can't build at the table. Nothing in it but two iron spheres, the same size, the same colour, the same everything, some distance apart. Name a difference between them. There isn't one, and there are still two of them. People have argued about Black's spheres ever since, and I'm not going to settle it.

I don't have to, because Leibniz's claim is about *every* property a thing has. The theorems above are about three: the line and the two counters. The two runs aren't indiscernible in his sense at all: one has a fact in its table and its flag up, and the other doesn't. They're indiscernible to a window, which is a much cheaper thing, and it's the thing that matters for a machine, because an observer of a machine only ever gets a window. Leibniz's gentleman would always find his difference if he could look at everything. It doesn't help him if all he's allowed to look at is three things about each leaf.

There's a horse in this, too, if you squint. In Germany in the early 1900s a mathematics teacher named Wilhelm von Osten showed off a horse called Clever Hans, who answered arithmetic questions by tapping a hoof, and got them right, in front of crowds. Hans gave the same answers out as a horse that could count, and that's the idle run's matching window. Then the psychologist Oskar Pfungst stopped looking at the answers and started changing the conditions, and published what he found in 1907. Put blinders on Hans, or have him asked by someone

who didn't know the answer, and the arithmetic fell apart. Hans was reading the questioner: tiny, unintended movements of the head and body as the hoof reached the right number. That's the probe. Pfungst didn't argue with the answers. He asked a question the answers couldn't fake, one that needed the real thing to be there, the way a COMMIT needs a checked fact to be in the table: run COMMIT *is zero* A after each of the two runs, and one passes and the other traps (Theorem 1.15). And the part of the story I like best: nobody was lying. Von Osten was never persuaded by Pfungst and kept showing his horse. A matching window doesn't need a cheat behind it, which is why you want a probe. An accusation is the wrong tool for it.

The rebar example, on the actual counters
Put a one in counter A, leave B at zero, and give the machine these four instructions:

$$\text{CHECK } (A \geq 1); \quad \text{COMMIT } (A \geq 1); \quad \text{CERTIFY}; \quad \text{DEC } A \text{ 5}.$$

The check passes. The claim gets committed and certified. Then DEC takes the one away. Look at the counters now: zero and zero. The value that passed the check has gone.

Start again with both counters zero and give the same instructions. This time the first check fails and the machine traps. The zero stays where it was. Look at the counters at the end: zero and zero again. One run has its certificate up; the other has it down.

□ **Theorem 1.17** (Same final counters and bill, different certification)**.** For those two clean starts and that word, both final counter pairs are $(0, 0)$ and both ledgers are 3. The first certificate is true and the second false. No function of just the final counter pair can recover which of those starts passed the check.

Proof. For the start $A = 1, B = 0$, CHECK sees $1 \geq 1$, writes the edition-zero fact, advances the line from 1 to 2 and makes the ledger 1. COMMIT finds that live fact, puts it in the channel, advances to line 3 and makes the ledger 2. CERTIFY sees the occupied channel, raises the flag, advances to line 4 and makes the ledger 3. DEC sees a positive A, changes it to zero, advances to line 5 and raises A's version from 0 to 1. Its price is zero. The final counters are $(0, 0)$ and the flag is up.

For the start $A = 0, B = 0$, the first CHECK fails. It sets the trap, leaves the line at 1, leaves the table empty and charges one. The external word still supplies COMMIT and CERTIFY: on a trapped state they change no core field and cannot raise the flag, but each adds its unit to the ledger. DEC then changes nothing and charges zero. The final counters are again $(0, 0)$, both versions stay zero and the

flag is down. Both ledgers are 3. A function of the common counter pair would have to return both flag values, which is impossible. ■

The counters show the same surface. The record keeps the difference. Open the rest of the box and the trap, version and line number tell different parts of the story too: one run reached DEC, while the other stopped at the check.

Ask the successful run to CHECK $A \geq 1$ once more. It fails; A is zero now. The old certificate stays up because the earlier certification still happened. That receipt keeps its past even while the workspace goes on to something else.

One clipboard, 65,535 different publication histories

Open the whole native box this time. Read the counters, versions, table, channel, line number, trap, ledger and lamp. I can make all of them agree while the files of published claims differ in 65,535 ways.

Start clean at $A = 16, B = 0$. CHECK the sixteen claims $A \geq 0, \ldots, A \geq 15$. Those counter values are initial data, before any instruction: versions zero, line 1, ledger 0, empty table and channel, and trap and flag down. Every one passes on edition zero. The clipboard now has sixteen entries. The line number is 17, the bill is 16, and the certificate is still down. I can choose which of those checked claims to publish.

Visit the claims in order. At each one, choose between these blocks:

publish it:	COMMIT claim; CERTIFY,
leave it unpublished:	COMMIT claim; COMMIT claim.

Count the steps in either block: two. Count the charge: two. Look at the channel when it finishes: the same claim. The counters and table stayed put. Once one publication has raised the lamp, that lamp stays up whichever later blocks you choose. CERTIFY reads the channel without consuming it; this is why the two blocks leave the same channel as well as the same line and bill. The predicates overlap logically; it is the choice to publish each checked claim that is independent.

Theorem 1.18 (The exact native publication-history fibre)**.** In the extension retaining the set of published claim/edition facts, exactly 65,535 different nonempty sets can lie above one reachable native endpoint. This is the maximum over clean native runs with cert true. Sixteen additional fixed binary bits suffice to encode the set relative to the native table, and fifteen do not suffice in the worst case. □

Proof. First prepare the table. After the first j checks, for $0 \leq j \leq 16$, its entries are exactly $A \geq j-1, \ldots, A \geq 0$, newest first and all on edition zero (the table is empty at $j = 0$). The line is $1 + j$, the ledger is j, the counters and versions are unchanged,

the trap is down and the flag is down. This holds at $j = 0$. If $j < 16$, there is still room and $16 \geq j$, so the next CHECK prepends precisely the next fact, advances one line and charges one. That proves the preparation claim through $j = 16$.

Now process the sixteen facts in order. Each COMMIT finds its fact in the unchanged table at the current edition. Either block ends with that fact in the channel, advances two lines and charges two. The publishing block additionally raises the flag and inserts that fact into the archive; the other block changes neither. Inducting on the number j of blocks gives line $17 + 2j$, ledger $16 + 2j$, and exactly the chosen subset of the first j facts in the archive. The flag is up if and only if at least one publishing block has occurred. None of the blocks changes counters, versions, table order or trap. At $j = 16$ every nonempty choice therefore has the same native endpoint: $A = 16$, $B = 0$, versions zero, line 49, ledger 48, trap down, flag up, and channel $(A \geq 15, A, 0)$. Its archive is precisely its chosen subset. There are two independent choices at each of sixteen positions, hence 2^{16} subsets; excluding the empty one leaves 65,535 different archives above that endpoint.

For the upper bound, keep two invariants together: every published fact is in the table, and every occupied channel names a fact in the table. Both hold at a clean start. CHECK only prepends to the table. A successful COMMIT selects an entry already there. CERTIFY publishes only its occupied channel. Counter writes can make a fact stale; they preserve the channel and never remove table entries. Failed and trapped instructions do not introduce a new channel fact or publication. Thus both invariants survive every instruction, including later writes. Every final archive is a subset of the final table.

If that table has d distinct entries, it has at most $2^d - 1$ nonempty subsets. The table has at most sixteen entries, so $d \leq 16$ and $2^d - 1 \leq 65{,}535$. An explicit code puts beside each table entry one bit saying whether it belongs to the archive, and pads with zeros to sixteen positions. Duplicate entries receive matching bits. Read back the entries with bit one and you recover the set. Equal codes therefore give equal archives, and an archive above a flag-up state uses a nonzero code. Finally, fifteen fixed bits have only $2^{15} = 32{,}768$ values, fewer than the exhibited 65,535 archives; sixteen bits have 65,536 values and the code just given suffices. ■

Put sixteen checkboxes beside the clipboard's sixteen lines. A tick says that line's claim was published at least once. The table already names the claims and editions; the extra marks say which ones made it into the publication file. The lamp gives the same yes for every nonempty selection. Open the checkboxes and all 65,535 selections come apart. Each tick answers whether that claim was published at least once. Order and repeated publications leave these answers unchanged.

In the gauge counts of Section 15.3 the camera showed the working value. Here I let it see the entire native machine and still found histories it could not recover.

The map here forgets the added publication file; its fibre contains those different files above one native endpoint.

The sixteen-line clipboard eventually fills. Even sixteen repeated checks of one fact fill it, and the next CHECK traps. The unlimited-edition machine has a larger obligation: keep accepting independent publication choices across every finite number of editions and keep them recoverable at the same working picture. Its fibre has to keep growing. That is the count from Section 15.3, with its own camera and its own task.

1.10 The exact price is three

Section 2 said a raised flag costs at least one. On this machine that's true and too modest. A raised flag needs a CERTIFY, a CERTIFY needs a COMMIT, a COMMIT needs a CHECK, and each of those is its own instruction costing one. So a raised flag costs at least three. One of what: three units of μ, one per paid act, each one the least price a record-layer merge allows. And three is the number of links in the chain, not a guess that happens to hold.

Observation 1.19 (The minimum price of a certificate)**.** *If* s_0 *is clean and* $\mathrm{cert}(\mathrm{run}(t, s_0))$ *is up, then* $\mathrm{cost}(t) \geq 3$ *and* $\mu(\mathrm{run}(t, s_0)) \geq \mu(s_0) + 3$*. For programs: if the flag is up after* n *program steps from* $\mathrm{start}(a, b)$*, then* $\mu \geq 3$ *there.*

Proof. Theorem 1.11 finds a CHECK, a COMMIT and a CERTIFY at three different places in t, each costing 1, and every other instruction costs at least 0. The ledger statement is Theorem 1.6. A program run is the run of the instructions it actually executes, so the program form follows. ∎

The three is three independent units, one for each part of a clean start. A flag that is down buys the first: a run that raises it costs at least one. An empty channel buys the second: with the flag down and the channel empty, a run that raises the flag costs at least two. An empty table buys the third, and that is the theorem above. Leave out any one part and its floor fails, and a cheaper certificate is there: with the flag already up it costs 0, with a commitment already in the channel 1, and with a live fact already in the table 2. Every run from a state that ends with the flag up costs at least three exactly when the flag is down and one of two things holds: the trap is up, or the channel is empty and every fact in the table is about an older version of its counter. And three is paid exactly from every clean start whose trap is down.

□ **Theorem 1.20** (Three is tight)**.** For all whole numbers a and b, run the program CHECK *is zero* A; COMMIT *is zero* A; CERTIFY for 4 program steps from start(a, b). If $a = 0$, it ends with the flag up and $\mu = 3$ exactly. If $a \neq 0$, it ends with the flag down and the trap up, and for every n, n program steps from start(a, b) leave the flag down.

Proof. If $a = 0$, the checker for Zero says yes at 0, so the first instruction adds $(\text{Zero}, A, 0)$ to the empty table and charges 1. No counter changes, so the second instruction finds that same fact, puts it in the channel and charges 1. The third finds the channel occupied, raises the flag and charges 1. The line is now 4, beyond the three-instruction program, and the fourth program step adds no charge. The final ledger is exactly 3. If $a \neq 0$, the checker says no, so the first instruction sets the trap and leaves the flag down. A program run takes no step once the trap is up, so every ■ later program step leaves the state, flag included, exactly where it is.

Theorem 1.20 is the worked run from early in this chapter, now holding from every start.

The version whose check can't fail, CHECK *at least 0* A; COMMIT *at least 0* A; CERTIFY, certifies from every start for exactly 3. So the floor of three is reachable from every loaded start, and that's the machine declining to police trivial checks. On that program the check distinguishes no inputs.

The floor is 3, and a run whose check could have failed pays exactly 3, so 3 is the price. The bound has no slack in it.

Don't mistake that 3 for a measure of how hard a claim is to certify. It's the same 3 for every claim, easy or hard, because only the three acts that make a claim are priced and the counters work out whatever the check needs at no charge on the ledger. A cost of certifying that grows with the claim would have to price the checking itself, and that's a different question with its own literature. Four theoretical computer scientists, Guy Blanc, Caleb Koch, Jane Lange and Li-Yang Tan (2022), studied one version of it under the name the query complexity of certification: how many questions an algorithm has to ask a black-box function to hand back a small certificate for its answer. Their count moves with the function. Mine doesn't move at all.

And that's every piece of the machine, standing on six instruction kinds: a base that computes everything two counters can, a ledger that's exact, a toll the universal floor applies to, a flag that's a latch of the base, a window that can't see the record, and, from a clean start, a commitment that can't happen unless a check of the same claim, on the same edition, came first and was right.

1.11 The machine versus the program it runs

The two universal programs of Section 3, U and U_P, come down to one distinction, and it's worth saying on its own, because the two halves are easy to run together: *the machine doing the stepping* (what the machine is) and *the program it runs* (what runs on it). The toll of Section 2 is a property of the machine doing the stepping. A rule a program keeps holds as long as the program keeps it.

I keep a picture for it: play Monopoly on the counter of a real bank. The game has a bank of its own, with its own rules, and one of them is printed in the rules sheet: the Bank never goes broke, and if it runs out of money the Banker may issue more by writing on any ordinary paper. Inside the game that's fine; the players agreed to it. The real bank's teller, standing right there, has no opinion about any of it. Push a thousand in Monopoly money across her counter and her ledger doesn't move, because her rules are about her own money. The game's money is nothing to them. The real bank is the machine doing the stepping. The game is a program it happens to be hosting. A rule that lives in the game holds as long as the players keep to it. A rule that lives in the bank holds whatever the players do.

(And since the game is on the table: its box told an origin story for decades, that an out-of-work salesman named Charles Darrow invented it during the Depression. The board is older. Elizabeth Magie, an American writer and game designer with an economic argument about land to make, patented The Landlord's Game in 1904, with a square track, railroads and a square that sends you to jail, and in 1902, in *The Single Tax Review*, described it as "a practical demonstration of the present system of land-grabbing with all its usual outcomes and consequences." Darrow sold his version to Parker Brothers in 1935, and the company bought Magie's patent for a few hundred dollars, five hundred by the accounts I found. A story told by the company selling the product is a lead to check, every time. It changes nothing about banks. It's a good reminder of who writes the story on the box.)

A fixed machine can enforce an invariant in its own transition table, and that includes a Turing machine. Give its tape a ledger track and a certification cell, and write a table that never sets the cell without bumping the ledger: that machine keeps the invariant in its step relation. The ledger track is part of its state, though, and a ledger kept in the state isn't a price on the move. The toll of Definition 2.2 puts the price on the move, the same from every state, so what it charges depends on what the machine's moves are. Give it one move, the next step its table picks: if the table sets the cell from any configuration at all, that move costs at least one, so every step does, whatever the track says, and the machine is a clock. Give it a move for each row of its table: every row that can set the cell costs at least one wherever it's taken, and a row that never sets it can cost nothing. Either way it's the price on its moves that meets the toll, and the track only simulates it. Priced

that way, it obeys the toll. If its table never clears the cell and some run sets it, it's a Thiele machine itself, so it can't be a counterexample to the model. What it doesn't do, built the ordinary way, is enforce the toll on the programs it runs. A universal machine whose table was written only to interpret carries the simulated certification bit as plain data on its tape. If the host is programmable, so that other code shares that tape with the guest, any of that code can flip the bit without paying, because the host's step has no opinion about it. So an ordinary programmable computer, as built, doesn't enforce the toll for its programs; that comes from how it's built, and it could be built to enforce it. A host whose own step mirrors the guest's bit and charges for its crossing enforces it, and the universal programs U and U_P are fixed tables that go one better: they earn their own flag, by their own check, commit and certificate, exactly when the guest's would rise, and pay on their own ledger what the guest pays, U exactly and U_P to within a surcharge of at most two when the guest's reading starts at no (Theorems 3.5, 3.4, 3.7 and 3.8). That's a statement about which reading a step rule prices, not a claim that Turing machines can't price one.

Computing ran into this exact gap in hardware, and how it got out is worth the detour. In 1974 two computer scientists, Gerald Popek and Robert Goldberg, asked when a machine can run guests, a whole operating system inside another, efficiently and safely. Their answer is one sentence you could check against any instruction set. Call an instruction *sensitive* if it reads or changes the machine's own settings, the things the host has to keep for itself, and *privileged* if running it outside the host's own mode makes the machine stop and hand control to the host, which is called a trap. For the kind of machine they modelled: if every sensitive instruction is privileged, a host can be built. The guest runs at full speed, and whenever it touches something that matters, the machine itself stops it and lets the host decide, and a host like that is one whose step has an opinion.

Then the obvious next question: did the machines people actually bought pass the test? The most common one didn't. In 2000 John Scott Robin, an Air Force officer, and Cynthia Irvine, a computer-security researcher at the Naval Postgraduate School, went through every instruction of Intel's Pentium and found seventeen that are sensitive and not privileged; POPF, which loads the processor's flags, is the one usually pointed at. A guest could run them and the host would never hear about it. The fixes came two ways: VMware rewrote the guest's code on the fly so the troublesome instructions never ran as written, and in the middle of the 2000s Intel and AMD added hardware support so they'd trap. Does that change anything here? It doesn't change the theorems. Popek and Goldberg were after speed and isolation. A record and a price weren't their question, so their result isn't mine and mine isn't theirs. It changes the picture, though. The universal programs sit where they'd want a host to sit: the host's own steps collect each guest's toll and earn each guest's

record again. And an industry learned the same lesson the expensive way: a rule binds the guest only if it lives in the step underneath.

The smallest version of "a ledger beside an ordinary machine" is a Turing machine configuration with a counter next to it, where each step adds one, plus one more if it writes a nonblank symbol. The counter doesn't disturb the machine.

Theorem 1.21 (A counter beside a Turing machine changes nothing)**.** Let a Turing machine with transition table δ step a configuration c to $\mathrm{tm}_n(\delta, c)$ after at most n steps (stopping when δ has no entry). Pair each configuration with a counter, and step the pair by stepping the configuration and adding the step's charge. Then for every n, δ and pair (c, m), the configuration part after at most n steps of the pair is $\mathrm{tm}_n(\delta, c)$. In particular this holds from $(c, 0)$. □

Proof. Put the ordinary run and the run with the counter side by side. Both start with configuration c. Whenever their configurations agree, they look up the same entry in the same transition table. If the entry is absent, both stop. If it is present, both take the same next configuration; only the paired run also changes its counter. Their configurations therefore agree again, ready for the next comparison. This covers every step up to the chosen limit n, as well as a stop before that limit. The counter's value never enters a lookup. ■

The pair is a Turing configuration with a number riding beside it, and nothing more. Its counter is kept in the state, and a ledger kept in the state isn't a price on the move: Definition 2.2 puts the cost on the move's name, and this counter's charge depends on what the step writes. This theorem therefore does not supply exact accounting for the basic instruction-price signature, or a reading with an earned-history meaning. It does not rule out equipping the same state space with other instruction prices and a Boolean reading satisfying the basic toll. The projection claim is simply that this additional count leaves the tape computation alone.

One limit on all of this needs saying, so it isn't read as more. Certification is one event. If you wanted a machine to carry some other kind of reading, you'd need an event for it, evidence for it and a price law for it, and nothing here supplies any of those.

1.12 The small machine's merges

And the whole small machine isn't the eight-state piece it runs. It merges some states without adding anything to the ledger.

□ **Theorem 1.22** (The small machine's merges)**.** (1) A move of the small machine that takes the flag from down to up is CERTIFY; CERTIFY merges two different full states into one (a state with the flag down and the same state with the flag up land on the same state), and it costs at least one. (2) DEC A 2 also merges two different full states, and it costs nothing. So it is false of the small machine that every move that merges is priced.

Proof. (1) Only CERTIFY changes the flag, and it costs one. The two states with the flag down and up, and a committed claim, are the same state after CERTIFY. (2) Take a core at line 1 with A at 1 at version 0, and the same core with A at 0 at version 1. DEC A 2 takes the first to $A = 0$ at version 1, line 2, a write; it takes the second to line 2 with A still 0 at version 1, because DEC on zero is no write. Same state, zero cost. ■

The theorem puts names to the last item on the list of what this machine doesn't do, near the end of this chapter. Part I's counting of states wants a finite machine that puts a price on every merge. This machine has counters with no top, and it prices the three record acts and leaves its counting merges at a price of zero, so Part I's counting of states doesn't reach it whole, and that's why the eight-state piece is a piece. The record's part of the state closes the rest: asked only of merges between states with the same line and counters, the merge premise holds for this machine's own prices and forces the toll on it with no finiteness premise (Theorem 5.1).

1.13 Reading the instruction rules

The instruction rules settle some concrete questions about running this machine. Follow them through a check, a zero decrement and a trapped trace.

- **The check price is flat.** CHECK costs 1 whether it's asking "is this 0" or "is this at least a billion." The property language is fixed and each property is one evaluation, so "pays for its work" here means that flat price, the same however hard the question was. A machine that charged by the difficulty of the check would be a different machine, and I haven't built it.
- **DEC on zero isn't a write.** It doesn't change the counter, so it doesn't change the version. That's the right call for versions (nothing changed), but it means DEC on zero is a way to move the line number that adds nothing to the ledger and leaves no trace in the record. The idle run in the subsection on the window, earlier in this chapter, uses exactly that.

- **After a trap, a trace still pays.** On programs, a trap stops the run. On arbitrary traces, instructions after a trap are each charged their cost and none of them changes the core. That isn't a separate choice. A price belongs to the instruction, the same from every state (Definition 2.2), so a CERTIFY on a trapped machine costs what every CERTIFY costs, and the ledger adds it; Theorem 1.6 has no exceptions for that reason. The choice is one level up, in putting the price on the instruction and not on the instruction at a state, and what it brings with it is that a trapped machine is still charged for instructions that change nothing.
- **"Checked" means what the property language can say.** On this machine that's *is zero*, *is even* and *is at least* n, and a live fact means one of those holds of one counter's current value. It doesn't mean the program is correct, or that anything follows from the property, or that the property was worth checking. So can a proved machine of this kind check "sorted," the claim the sorter-and-copier table of Section 15 runs on? It can, through an encoding. A copy of this machine, instruction for instruction, with its property language left open, has provenance, checker soundness, no forging and the floor of three proved once for every property language that comes with an exact equality test and a checker proved equal to its meaning. Give it "the list this counter encodes is sorted" as a property and it certifies a sorted list for three and refuses an unsorted one forever. The instruction rules are those already given, with the chosen property language in place of the three original properties. For sorted lists, decode 0 as the empty list and $2^h(2t+1)$ as the list with head h and tail decoded from t. Every positive code has t smaller than itself, so decoding terminates. The checker returns yes on an empty list or singleton, and on $a, b, \ldots$ exactly when $a \leq b$ and it returns yes on $b, \ldots$. Induction on the list proves that this is exactly adjacent nondecreasing order. The provenance proof uses only equality of properties, and soundness only that the checker's yes means the property holds, so the same instruction-by-instruction argument applies. What makes that a check of the list is the proof that the checker means sortedness. Without that proof, "sorted" would be a number some unconstrained code wrote into a counter, and the record would be honest about the counter and silent about the list.
- **The certificate is historical.** Between the COMMIT and the CERTIFY, the counter can change, and the flag still goes up. The flag says a claim was checked and committed about an edition. It says nothing about whether the claim is still true.
- **One latch, and a channel that moves on.** The flag records that an earned chain happened, once. After that it stays up, and later CERTIFYs do not change the flag; they still charge the ledger and, when successful, advance the line. The channel is the part that names a fact: with the flag up from a clean start, it names a fact that some earlier passing COMMIT committed. But a later commitment replaces

it, so at the end it names the last earned commitment, which needn't be the one the flag rose on. Two runs from start$(0, 0)$ show it, worked from the instruction table. First run: CHECK *is zero* A; CHECK *is zero* B; COMMIT *is zero* A; CERTIFY; COMMIT *is zero* B. Second run: the same five instructions with the last two swapped, so COMMIT *is zero* B comes before CERTIFY. Both end in the same state: both counters 0 at version 0, both facts in the table, the channel holding (*zero*, B, 0), the trap down, a ledger of five and the flag up. In the first run the flag rose at step four, on the commitment about A. In the second it rose at step five, on the commitment about B. So the flag is an ever-certified latch: it says an earned chain happened at some step, and the end state doesn't say which claim or which step. A receipt naming the claim the flag first rose on would need that claim kept where nothing can overwrite it; a receipt with a time on it would also need the step number kept; and several certifications that each last would need a record richer than one flag. The publication file constructed earlier in this chapter keeps the set of claims certified at least once. That extension shows exactly what the native state left out. It still does not record which certification came first or at what step; those questions would need their own retained answers.

- **Sixteen checks per run, ever.** Facts never leave and the table holds sixteen, so a run gets at most sixteen passing CHECKs in its whole life, and the seventeenth traps. A certificate needs only one, so this costs nothing for the theorems, but it's a hard wall for any program that wants to keep checking forever, and it's this machine's small version of "you can't keep all of it" from Section 15. A CHECK that repeats a claim already in the table, same property, same counter, same edition, passes again and takes another entry. So the wall is sixteen passing CHECKs, and sixteen different claims is a different count. Removing duplicates, evicting old facts or collecting dead ones would each be a different design, and each would need its own invariants proved.
- **The clean start matters.** Provenance, soundness, no forging and the price of three are all stated from a clean start. Hand the machine a table pre-loaded with facts nobody checked, and of course none of that follows. The theorems are about what the machine produces. What you smuggle in at the door is outside them. The door is narrower than clean for some of them: a commitment's provenance and checker soundness use only the table, and hold from any start whose table holds nothing but facts about older editions; the empty channel and the lowered flag come in for the flag's provenance and the price of three. Spell the door out once: the theorems cover any list of instructions anybody can issue, from a clean start, on a step that runs as written, with nothing else writing to the machine. They say nothing about whoever sets or restores the start, about a reset, about a device that misbehaves, or about whoever carries the flag's value to you.

- **The universal programs have stated guests.** U (Section 3.1) runs every one of this machine's programs from every clean start start(x, y), with halting, answers, flag and ledger matched (the flag over the whole run, the ledger at matching points). U_P (Section 3.2) runs every computably presented machine, with its reading decided by a program, its record carried as a latch, and its ledger matched to within a surcharge of at most two when its reading starts at no, and of three when it starts at yes. When the guest certifies with a ledger below three, no program on that host, run from a clean start, matches its ledger exactly. A machine outside that class, with a reading no program decides, isn't covered by either. The host with a guest step built in, charging one per guest step whatever the step was, would have a ledger at least the guest's and usually more; U's is exact, and U_P's is exact up to the surcharge.
- **Encoding and running time.** The universality result takes its inputs already packed into the counters. Getting there from an arbitrary Turing machine goes through Minsky's exponential packing, and that packing is what it takes to reach two counters this way. Anyone who wanted this to run fast would have to start the analysis over.
- **The step charges for the act, not for what the claim is worth.** CHECK *at least 0* passes on every value, so CHECK, COMMIT, CERTIFY on it raises the flag for three from any fresh loaded start (the note after Theorem 1.20), and that's by design. The step checks order, aboutness and an honest checker. What a claim is worth is the entitlement count of Section 10, a number computed over the rivals a run narrows. A step sees one state, so it cannot compute that count. A check that cannot fail narrows nothing and costs three, and the book keeps both numbers, apart.
- **The toll on this machine comes from the record's part of the state, with no finiteness premise on the whole machine.** Section 3's counting of states forces a toll on a finite machine that prices every merge. This machine is neither. Its counters have no ceiling, as a machine that runs every program has to have, and its editions can't have one either (the version rule says why), so it's infinite. And, by design, its unpriced instructions merge states: take a core at line ℓ with c at 1 and version w, and a core at line $j-1$ with c at 0 and version $w+1$, everything else alike, and DEC c j sends both to line j with c at 0 and version $w+1$, at a price of zero. A price of zero doesn't mean the hardware pays nothing. Any physical heat belongs to the implementation's account. A positive floor requires the thermal premises and an input distribution with entropy loss; merely taking a step establishes neither. This ledger does not measure that heat. The ledger leaves those merges out on purpose. It counts the keeping in the record, not what computing spends. Computing doesn't have to merge; a reversible base

computes by paying in fresh states. On this machine full merge pricing would charge DEC, an ordinary step, and the chosen certified-machine schedule asks for computing to cost nothing. Charging every move would instead give a clock's schedule. Its total counts moves, rather than isolating record acts. The machine can still retain a record; this total alone does not say which acts established it. So merge pricing on this machine applies to the record's part of the state only: a move that sends two different states with the same line and counters to one state costs at least one. This machine's own prices meet it. The endless counters would allow a raise that merges nothing, and the flag doesn't take it: CERTIFY lands a state with the flag down and the same state with the flag up on one state, a merge of exactly that kind, in plain view. So every price list that meets the premise charges the flag-raising step at least one, with no finiteness premise, and through the trap it charges every CHECK and COMMIT at least one too, so a certificate from a clean start costs at least three on every such list (Theorem 5.1). That's a price I assume for losing distinctions in the record, and assuming it is all it is. Turning it into heat takes the physical conditions of Section 3; a unit on a ledger isn't heat on its own. The exact price of one, the least the premise allows on each of the three acts, is still a design choice.

1.14 What this chapter adds up to

So this chapter has one machine, and it isn't the model. It's a Thiele machine in the opening's sense: the toll is proved, its flag never comes down once it's up (Theorem 1.6), and from a clean start a run raises it (Theorems 1.20 and 1.7), and its toll charges the smallest kept distinction: raising its flag is a merge in the record's part of its state (Theorem 5.1). Its flat part computes whatever a Turing machine can, under Minsky's 2^n encoding, done outside the machine, its computing adds nothing to the ledger and can't touch the record, and, from a clean start, its record is earned.

If you ever find yourself wondering whether a property of this machine is the model's or just the machine's, hold it up against Section 2's definition of a Thiele machine, the latch of Section 12, and the four things certified asks for (Section 15.16). What follows from those holds of every machine that meets them; the rest (a table of sixteen facts, three properties) is this machine's choice. The flat price of one is the third of the four, and at least one of it is what the record's merge premise asks of each act on this machine. There are six instruction kinds, and a seventh, PAY, for a host that has to charge somebody else's prices (Section 3.2). If the model needs more than the five parts, the toll, a record that never falls and a run that raises it, this is where it will show.

CHAPTER 2

How far it reaches

The small machine earns its flag, and pays for it, on its own programs. Everything in this chapter asks how far that carries. Every statement in it is proved, and the runs are reported as runs. I give each statement here; the appendices explain the constructions and identify the companion files containing the full universal programs and simulation proofs, because they're about the reach of the case, not the case.

Other machines as guests. One fixed program, U, written in the small machine's own six kinds of instruction on a host with more counters, runs every program of the small machine as a guest, from every clean start. It fetches, decodes and carries out each guest instruction itself, the way Turing's universal machine does; nothing in the host means "run one guest step." It halts exactly when its guest halts, with the guest's answer in its counters; its flag goes up during its run exactly when the guest's goes up during the guest's, by its own check, its own commitment and its own certificate; and its ledger equals the guest's at every point where the two runs match. The host is a certified Thiele machine, and whether U halts is undecidable. Add one move, PAY, that costs one and does nothing else, and the same kind of program, U_P, runs every machine that obeys the toll and whose next move, step, prices and reading are computed by programs that always answer: it halts when the guest's run stops, its flag goes up exactly when the guest's reading is yes at some state of its run, on a chain whose check is the guest's own reading, and its ledger is the guest's prices plus a surcharge of at most two at the raise when the guest's reading starts at no, and exactly three when it starts at yes, a surcharge that no program on that host, run from a clean start, can always avoid. Section 3 has both.

Other shadows. Fit a record beside a register machine, a Turing machine or the lambda calculus, one that a run raises and nothing lowers, price the moves that raise it, and none of them computes one function more; what each gains is a record and a cost. As it stands, each is a shadow. With its record and its price column put back it's a Thiele machine, as long as it has a run that ends somewhere other than where it started (Theorem 2.3, Section 12).

Machines from parts. Wired side by side, end to end, or one inside the other, the toll carries over, and the tolls add the way money adds. What breaks, breaks for

reasons you could name in advance: a single flag for two claims costs two claims' worth, a clock is still a clock, a shared value without a version can't be trusted, and a hand-off that costs nothing can't be allowed to carry a certificate (Section 4).

CHAPTER 3

The universal machines

This appendix describes the two universal programs of Section 2: U, which runs every program of the small machine, and U_P, which runs every computably presented Thiele machine. It states their interfaces and results and explains the constructions. The instruction expansions and detailed simulation proofs are in the companion formal development; they are not reproduced in this section. The simpler host with a built-in guest step and the separate Rice and recursion constructions are not included here.

Where the complete constructions live

The following paths are relative to the accompanying repository. They identify the definitions behind the instruction counts and the proofs behind the reported results.

- `minimal/UniversalCodes.v`, `coq/kernel/foundation/UniversalBlocks.v` and `coq/kernel/foundation/UniversalLayout.v` specify the coding, instruction blocks and fixed list U. The layout establishes its length and paid sites.
- `coq/kernel/foundation/UniversalSim.v` and `coq/kernel/foundation/UniversalRun.v` carry the simulation through whole runs. The latter contains `U_simulation`, `universal_halting`, `universal_output`, `universal_flag_iff`, `universal_ledger_exact` and `universal_earned`.
- The corresponding priced-host files, in particular `coq/kernel/foundation/UniversalPLayout.v` and `coq/kernel/foundation/UniversalPRun.v`, give U_P and its guest simulation.
- `coq/kernel/foundation/PresentedUniversal.v` connects the compiled presentation to U_P. Its results include `presented_universal`, `presented_universal_exact`, `presented_universal_surcharge_le_two` and `presented_universal_no_exact_below_three`. The host-interface connections are in `coq/kernel/foundation/UniversalInterpreterLinks.v` and `coq/kernel/foundation/PricedHostLinks.v`.

Reading the statements below does not replace checking those expansions and proofs. In particular, the numbers 3728 and 3735 describe those specific program lists; they cannot be reconstructed from the informal handler descriptions alone.

3.1 One machine that runs them all

The small machine raises its flag only by checking, committing and certifying, and it pays for that. The next question is whether one machine can stand in for a whole class of machines, the way Turing's universal machine stands in for every Turing machine. What I mean by that is a *universal Thiele machine*: one fixed machine running one fixed program that runs any machine of a stated class as a guest, with every step of the guest's computing done by the host's own counter moves and every rise of the host's record put where the host's program put it. No program running on the host has to keep a convention for it. In full:

Definition 3.1 (Universal Thiele machine)**.** Fix a class C of machines that obey the toll, each with its runs from given start states. The class can hold machines that aren't Thiele machines, with readings that come back down or never rise. A *universal Thiele machine for C* is a Thiele machine H, the *host*, with one program P fixed for the whole class, a way to load any guest M in C with any start s_0 into a start state of H, and a way to read a guest's state off the host's registers, such that: (1) H is a Thiele machine whose counter moves cost nothing and can't touch its record, and those are its computing moves; (2) the run of P from the loaded state halts exactly when M's run from s_0 stops, and then shows the observable part of M's last state, as the class names it, its reading kept as a latch, and its ledger up to the surcharge; (3) the run passes, in order, through host states that show the observable part of M's states along its run, at which the host's record equals M's reading kept as a latch and the host's ledger equals M's ledger plus a surcharge, never negative and bounded by a number fixed for the class; (4) *the rise is the program's*: from every untrapped state, a record move selected by P either traps, or leaves the flag up exactly when it was already up or the current line is marked as raising by P. With the flag down and the move succeeding, that line decides whether it rises; (5) H is a *register host*: its counter moves are INC, DEC and HALT on registers, its record moves leave the registers alone and go to the next line or trap, and which move comes next depends only on the line and whether the trap is up; (6) each guest is loaded with the record down, no trap, a fixed line and the guest written into the registers, and the guest's state is read off the registers in a coding fixed with the class, before any guest is named. A *certified* universal Thiele machine also has its host certified, with clean loads, and its record, whenever it's up, standing on the host's own check of the guest's own justification, as the class supplies it.

For U the guest's justification is its chain claim; for U_P, the guest's reading at a state of the run. For U the observable part I've proved it matches with all six clauses is the guest's two counters, and in order it matches everything but the guest's version numbers, which no run of the guest can show; for U_P it's the whole state.

The guest lives only in the loaded state; the machine and the program are fixed before any guest is named. For U the surcharge is zero; for U_P it is at most two when the guest's reading starts at no, and exactly three when it starts at yes, so three bounds it over the whole class. Both are certified universal Thiele machines, by the theorems of this section put together: each meets the six clauses, each host is certified, with clean loads, and each one's flag stands on its own check of the guest's justification.

The two that U_P adds once a guest's record is up is, at worst, the price of certification, not of universality. A certified host raises its record from a clean load only after three marks, so on a guest whose whole ledger is one when its reading first says yes it falls two behind at least. A host without certification can keep up exactly: on a host with registers, INC, DEC, HALT, a paid move TICK that always raises the record, a paid move PAY that never does, and a paid move MARK that raises it when register 0 is at least 1, which no program below runs and which a record that tells starts apart needs, since without it no run's record depends on where it started, the program "count register 0 down to zero; TICK; HALT" runs the guests that count a pile down to zero and then raise their reading at a cost of one, each from its own pile, with surcharge zero. A guest whose reading is already yes at its start costs any host loaded with its record down at least one, and one is reached. And from a start at no one host keeps up exactly on the whole class: U_P's compiler on that host, with the three-act chain at the first yes replaced by one TICK and the rest of that move's price paid in PAYs. The toll on the guest makes that move cost at least one, so the TICK always fits inside it, and the host's ledger is the guest's at every matching point. On a host without certification the surcharge is zero from no and one from yes, and neither can be lowered. (And PAY is needed on such a host: a guest that pays while its reading is down makes the host run a record move that doesn't raise, which is what PAY is.)

Each clause stops something. For clauses (1), (4) and (5) there's a host that meets the others and fails it; (2) and (3) together stop a host that ignores its guest; (6) I state in words below. Clause (1) stops a host that raises its record early and lowers it again before anyone looks. Clause (4) stops a mark that reads the registers, or a copy of the guest kept outside them, and raises when the guest would: the same move on the same line would raise from one state and not from another. Clause (5) stops a host with a move that takes one step of the guest. In the one I built that move costs nothing on the guest's unpriced instructions and traps on its paid ones; costing nothing, it would have to be a counter move, and a counter move

never traps. Make such a move cost one instead and it's a record move, which has to leave the registers alone, so it can't move the guest's counters, which (6) puts there. Clause (5) also stops a move costing nothing that fetches, decodes and carries out a guest instruction on the registers, because that move adds one to one register from one state and to another register from another, which no single INC or DEC does. Clause (6) makes the guest come in through the registers. It's the one clause I've stated in words without a theorem that it discriminates: a reading of the registers that adds part of the guest back in isn't excluded by anything I've proved. What stops a host that ignores its guest is (2) and (3), on any class whose guests differ in when they halt or what they record, which a universal class does. What the clauses allow is exact too. By (1), (4) and (5), the line, the registers, the record and the ledger of a run before any trap depend only on the line, registers, record and ledger it was loaded with: each counter move acts on those alone, each record move leaves the registers and moves the line, and its effect on the record is fixed by its line. So a host may carry anything else along, even a copy of the guest that its counter moves step, and what it carries along that way can't touch anything the host shows. One route I haven't closed: a move that traps may read anything, and stopping the run is all it can do. U_P never traps on its class, so for it the route is shut. For U it's shut too: U is loaded with every version at 0 and an empty table and channel, so whatever a trapping move of U reads was written by the run itself, from the registers it was loaded with. For a host in general it stays open.

None of this needs certification. U_P's own program, read on a host with no claims, no checker, no table and no channel but with that MARK, with CHECK, COMMIT and PAY read as PAY and CERTIFY as TICK, meets (1) to (6) with the same surcharge: U_P never traps on its class, and while nothing traps the two hosts agree, step for step, on the line, the registers, the record and the ledger. Certification adds the tie to the guest's own justification, and it needs (4) as much as anything: a host that carries out a hidden copy's own check, commit and certificate meets the tie and fails only (4) and (5).

And I want it the hard way. The easy way is to give the host a move that means "take one step of the guest," and then "it runs any guest" is true because I wrote it that way. That is a useful comparison, but it is not the construction given here. It is not Turing's kind of universal. Turing's universal machine has no instruction for "do what the machine on the tape would do." It has the same kind of instructions every other Turing machine has, and one fixed table, and the table does the reading and the deciding and the carrying out. The bar here is the same, and clauses (4) and (5) make it exact: one fixed program, every computing step a counter move, and every rise performed by that program. The table, channel and trap still govern whether record operations succeed; the restricted computing-step claim is not independence from all auxiliary state.

Turing's own explanation is worth having in your head, because the program below copies its shape. In the 1936 paper the universal machine's tape starts with the machine to imitate, written out as a string of letters, its standard description, and after it the imitated machine's complete configurations, one after another: where its head is, what state it's in, what's on its tape. To take one step, the universal machine finds the last configuration, looks through the description line by line, comparing, until it finds the line for that state and that symbol, and then writes out the next configuration the way that line says. Then it goes back and does it again. It leaves marks on squares so it doesn't lose its place. Every bit of that is ordinary instructions on an ordinary tape, and nothing in its table knows any particular machine.

Turing's universal program came first on paper, and one that people actually came to use arrived through the side door. John McCarthy, who had started the artificial intelligence project at MIT with Minsky, was writing up his new language Lisp around 1960 and wanted to show it was neater than Turing machines, so he wrote a universal function for it, *eval*, the language describing how to evaluate itself, and argued it was shorter and easier to follow than a universal Turing machine. In his own history of Lisp, that was for the paper, a thing to read; the notation for writing programs as data was made up for the paper with no thought of using it in practice. Then Steve Russell (S. R. Russell in McCarthy's account), a programmer on that MIT project, noticed that eval could serve as an interpreter and coded it by hand, and there was a running language with an interpreter that nobody had planned. McCarthy adds that the surprise froze some decisions he'd made "rather lightheartedly" for the paper. So the trick has a habit: written to be read, it ends up being run. The program below goes the other way round, on purpose. It's written to run, and the proofs show that what runs is what the page says.

The host. The program runs on a machine with the small machine's instructions and the small machine's prices: INC, DEC, HALT costing nothing, and CHECK, COMMIT, CERTIFY costing one each, with the same table of sixteen facts, the same channel, trap, ledger and flag. One change: it has a counter for every whole number, each with its own version, where the small machine had two. There's no new kind of instruction and no new price. Take away the table, the channel and the flag and what's left is old: a machine with as many registers as it needs, adding one, taking one away and jumping on zero, is the unlimited register machine of John Shepherdson and H. E. Sturgis, two logicians who proved in 1963 that such machines compute every partial recursive function.

Definition 3.2 (The host machine)**.** The *host machine* is the machine of Definition 1.3 with one change: it has a counter, with its version, for every whole number r (a *register*), and INC r, DEC r j, CHECK p r and COMMIT p r name one register each. Its property language has one property, Slot. Write $\langle m, n \rangle$ for the number

$(2n+1)\cdot 2^m$; every positive number is $\langle m, n\rangle$ for exactly one pair, and 0 is no pair. Number the small machine's properties Zero $\mapsto$ 0, Even $\mapsto$ 1, Ge(n) $\mapsto$ $n+2$. The checker of Slot says yes at a value x exactly when $x = \langle m, v\rangle$ and the small machine's checker says yes to the property numbered m at v; its meaning is the same with holds in place of the checker.

There's one place the guest's machine shows up in the host. The host's single property knows how to read the small machine's three-word property language: "this number is the pair (property, value) and the property holds of the value." It doesn't know the small machine's step. Knowing the language and not the step is the counter version of Turing's universal machine knowing the alphabet its guests write in.

Two facts that force the design. Why more counters? The answer is two facts, and the program looks the way it does because of them.

The first is about reading. On this machine a program that wants a counter's value in its own control flow has to take pebbles out (DEC) and put them back (INC), and every INC, and every DEC on a counter above 0, is a write, so the counter's version goes up. A fact is about one version, so a read done that way kills every live fact about the counter. Not every look is a write: CHECK reads a value without writing it, a DEC on 0 changes no version, and a fresh CHECK after such a read can make a new live fact. But the guest's counters are exactly the ones the host has to read by taking pebbles out, again and again, to decide which way a DEC jumps and to work out a property's answer, and each of those reads kills whatever fact was kept there. So the host can't keep the guest's facts on the copies it reads, and a copy it only writes is what the second fact rules out.

The second says that even if reading wrote nothing, a fixed program couldn't keep them by sending each guest property to one of a fixed, finite list of host properties and checking it on the same register the guest's fact is about. The theorem below rules out that and nothing more, and the pigeonhole is what rules it out.

□ **Theorem 3.1** (No host fact on an exact copy of the guest's counter)**.** Take any host property language with an exact equality test and any checker, a finite list Q of host properties, and any translation of the small machine's properties into members of Q whose host check passes whenever the guest's check passes on the same number. Then there are $n \neq m$, with Ge(n) and Ge(m) translated to the same host property, such that the guest program CHECK Ge(n) A; COMMIT Ge(m) A traps within two steps from every clean start, while on every host state with the trap down and room in its table, and some register holding a number at least n, the translated CHECK and COMMIT on that register both pass and put the committed claim in the channel.

Proof. There are infinitely many properties $\mathrm{Ge}(n)$ and only finitely many translations in Q, so distinct n, m share a translation q. From a clean guest start, a passing CHECK of $\mathrm{Ge}(n)$ inserts only that claim; COMMIT of the distinct $\mathrm{Ge}(m)$ therefore fails. If the CHECK fails, the guest already traps. On the host, a value at least n makes the translated CHECK of q pass by hypothesis. With room in the table it records q at the register's current version, and the following COMMIT of that same q succeeds. ∎

So the host keeps the guest's facts somewhere else: in *slots*, extra registers used for nothing but holding facts, one per checked fact, each holding the pair (property number, value) that the guest checked. The pigeonhole doesn't bite a slot, because the property lives in the slot's number. The instruction doesn't have to carry it: CHECK Slot on a slot asks about whatever pair is sitting there, and there are infinitely many pairs. A slot's number changes only when the program is moving a fact into it. And when the guest writes the counter a fact was about, the program gives every slot about that counter a nudge, one INC and one DEC, which leaves the number alone and pushes the version up, so the host's facts go stale at exactly the moment the guest's do.

For example, nudging a slot holding 18 changes it to 19 and back to 18. Its version increases twice, so an old fact still names the old edition. The simulation promise is at the completed handler's endpoint, after both writes.

Count what costs anything. Fetching, unpacking, copying, nudging and branching are all INC and DEC, and all cost nothing, as computing does on the small machine. The only paid instructions are the CHECK, COMMIT and CERTIFY sites, and each guest CHECK, COMMIT or CERTIFY makes the program run exactly one of them. So the host's ledger lands on the guest's, unit for unit. And when the guest's flag goes up, the host's goes up by the host's own CERTIFY, standing on the host's own COMMIT and CHECK on a slot. The host isn't copying a bit; it runs the same chain on its own registers.

Definition 3.3 (The program U, and loading a guest)**.** U is the fixed list of 3728 host instructions specified by the companion block expansions identified above, starting at line 1 and mentioning registers 0 to 80; exactly 69 of them cost anything. For a guest program P and a start (x, y), the *loaded host* $\mathrm{hload}(P, x, y)$ has register 0 at x, register 1 at y, register 2 at the number of P, register 3 at 1 and every other register at 0, every version 0, line 1, an empty table and channel, the trap down, ledger 0 and flag down. Write $\mathrm{grun}(m)$ for m program steps of P from $\mathrm{start}(x, y)$, and $\mathrm{hrun}(n)$ for n program steps of U from $\mathrm{hload}(P, x, y)$.

□ **Theorem 3.2** (U keeps pace with its guest)**.** For every guest program P, start (x, y) and number of guest steps m, there's a number of host steps n such that one of two things holds. Either the host is back at line 1 with $m \leq n$ and *related* to the guest: registers 0 and 1 hold the guest's A and B, register 3 holds the guest's line, the guest's trap is down, the two ledgers are equal, the two flags are equal, and the host's table and channel hold, fact for fact, slot facts matching the guest's facts and channel. Or both have stopped, with the same counters, the same trap, the same ledger, the same flag, and matching tables and channels. And from any related pair where the guest hasn't stopped, one guest step is matched by at least one host step that lands related again, or with both stopped.

□ **Theorem 3.3** (Halting and answers match)**.** For every P, x and y, the guest's run of P from start(x, y) halts if and only if U's run from hload(P, x, y) halts. And whenever the guest has halted at step m and U at step n, the host's registers 0 and 1 hold the guest's A and B, and the two traps, the two ledgers and the two flags are equal.

□ **Theorem 3.4** (The host's flag is the guest's, and its ledger is exact)**.** (1) The guest's flag is up at some step of its run if and only if the host's flag is up at some step of U's run. (2) For every m there's an n at which the two are related, with $m \leq n$, or both stopped, with $\mu(\text{hrun}(n)) = \mu(\text{grun}(m))$. And for every n there's an m with $\mu(\text{grun}(m)) \leq \mu(\text{hrun}(n)) \leq \mu(\text{grun}(m+1))$: in between matching points, the host's ledger never gets ahead of the guest's next step.

Equal ledgers at the matching points are a third sense of "exact," and the plainest of the three (Section 1 names all three): two ledgers holding one number.

□ **Theorem 3.5** (The host earns its own flag)**.** If the host's flag is up after n steps of U from hload(P, x, y), then the instructions U executed contain, in order, a passing CHECK Slot on a slot, then a passing COMMIT Slot on the same slot, with the slot untouched and its version unchanged in between, then a passing CERTIFY; the slot held $\langle$number of p, $v\rangle$ at the CHECK, with holds(p, v). And the guest's own run from start(x, y) contains a passing CHECK p c, then, with c untouched and its version unchanged, a passing COMMIT p c, then a passing CERTIFY, with c holding v at its CHECK.

Theorem 3.6 (The host is certified, and U's halting is undecidable)**.** (1) The host machine, with Slot as its property and one move per instruction, is certified in the sense of Section 15.16, over its own claim language, whose claims are "Slot holds of register r," whose meanings are written without the host's checker, and whose "unchanged" is "same value and same version of register r"; and U's run from $\mathrm{hload}(P, x, y)$ is a run of it. (2) Read as a Thiele machine it obeys the universal floor; a run of U that raises the host's flag costs at least 3 there. (3) U halts from $\mathrm{hload}(P, x, y)$ exactly when P halts from $\mathrm{start}(x, y)$, so the question "does U halt from this loaded host?" is undecidable in the reduction sense of Theorem 1.5. □

Put it together, plainly. One fixed program, 3728 instructions of the small machine's six kinds, with no guest step built into anything, runs every program of the small machine from every clean start $\mathrm{start}(x, y)$. It halts exactly when its guest halts, with the guest's answer in its first two registers. Its flag goes up at some step of its run exactly when the guest's goes up at some step of the guest's run, by its own check, its own commit and its own certificate. Its ledger is the guest's, unit for unit, wherever the two line up. And the machine it runs on is certified. Since the small machine runs every two-counter program, through those guests U runs every computation, under Minsky's 2^n encoding, done outside the machine. That's a universal Thiele machine for the small machine's programs, and the definition allows any other host that meets it; the next subsection carries it to every computably presented machine whose reading a program decides on every state.

There's fine print, and it comes before you take that anywhere. The guests start clean, from $\mathrm{start}(x, y)$; the whole-run theorems are about loaded hosts, and only the one-step theorem is about any related pair. The host's property reads the small machine's three-word language, so a guest with a different property language needs a different host property; the host of the next subsection has one, a single property that reads a universal claim language. Sixteen slots per counter is the guest's whole table size, reserved for each counter in case every fact is about that one. And it's slow: for every guest step it unpacks the program number from the front again, and it copies a number one pebble at a time. That's where its running time comes from. The theorems above say nothing about time; they match states and ledgers at the points where the two line up.

Enforcement in the step is the reference monitor's move from Section 15, Anderson's 1972 move: one mechanism everything has to pass through. What's put together here is a priced, earned record under that move, carried to every guest by one fixed program. The move itself is Anderson's. Kevin Hamlen, Greg Morrisett and Fred Schneider (2006) worked out, in their model, which policies mechanisms

like that can enforce at all, and their paper is where I'd go to see why enforcing a property of every run escapes the Rice-style walls that deciding one runs into.

Why make such a fuss about proofs that walk every case? Because the first universal machine had bugs. It's the machine in Turing's 1936 paper, and nobody gets to call Turing careless. In 1947 Emil Post, in an appendix to a paper on a problem of Thue, pointed out that the construction leaned on a convention it hadn't earned, and he patched a small defect in how the universal machine writes out each complete configuration. Around the same time Donald Davies, who joined the National Physical Laboratory in 1947, where Turing was designing a computer, worked through the universal machine's tables and found more mistakes, much to Turing's annoyance, as the accounts of Davies's life have it. (It's the same Davies who later named the *packet* in packet switching.) His corrections were finally printed in a collection edited by Jack Copeland, a philosopher and historian of computing who runs an archive of Turing's papers, *The Essential Turing*, in 2004. They may be some of the first bugs ever written, in a program for a computer that didn't exist yet. That's a lovely claim, and I haven't been able to confirm it, so take it as a lead. What it changes here: a universal machine is exactly the kind of construction where a careful person leaves a hole, *U* has 3728 lines for a hole to hide in, and each case of its simulation argument has to hold. The statements are on these pages; the complete definitions and case proofs are in the companion files named at the start of this section.

3.2 Every computably presented machine, on one host: driven, decided, priced in whole units, carried as a latch

U runs the small machine's programs, and through them every computation, under Minsky's 2^n encoding, done outside the machine. But a machine from somewhere else has its own states, its own moves, its own prices and its own reading, and running it through the two-counter encoding keeps its computation and nothing else. So the question Turing's machine raises for this book is the bigger one. Take any machine that obeys the toll and whose workings can be computed. Is there one fixed program that runs every such machine as a guest, keeping its halting, raising its own flag exactly when the guest's reading would rise, on an earned chain of its own, and keeping a ledger equal to the guest's own prices, with the host's flat ones nowhere in it? There is, within limits I'll list, and this subsection gives it.

First the class, said plainly. A *computably presented* machine is a priced record in Section 2's sense with a yes/no reading (states, moves, a step, a price for each move, a yes/no reading, and the toll: a move that turns the reading from no to yes costs at least one), together with a *driver* that picks the next move from the current state or says stop, and a way of writing states and moves as whole

numbers. “Computably” means four little programs, written in the plainest language of computable functions there is, the one built from zero, plus one, picking an argument, composing, recursion and search: one computes the driver, one the step, one the price, and one the reading. The driver, step and price routines are required to halt and give the specified result on valid state and move codes; their behavior on other numbers is not constrained by the presentation. The reading routine is required to halt on every number, returning the reading on valid state codes and no on other numbers. State and move encodings have left-inverse decoders, so distinct states or moves cannot share a code. Run from a start s_0, the driver's moves give one run. Its *ledger* after n moves is the sum of their prices, and its *latch* after n moves says whether the reading has been yes at any state so far. A computably presented machine needn't be a Thiele machine: its reading can come back down to no, or never rise at all. Every Thiele machine with a yes/no reading and such a presentation is in the class, so U_P runs those too.

The host gets one thing the small machine doesn't have: a paid move that certifies nothing. On the small machine every charge comes from a CHECK, a COMMIT or a CERTIFY, and those can charge without raising the flag: one passing CHECK and six COMMITs of the same claim cost seven. But every passing CHECK takes one of the table's sixteen entries for good, every COMMIT rewrites the channel, and a failing CHECK traps. So the host machine gets a seventh instruction kind, PAY: it costs one, goes to the next line, and changes nothing else; it never raises the flag, takes no table entry and leaves the channel alone. That's why the compiler uses it.

How far the small machine gets without it is proved, and the count is short. Take a run that never traps and ends with the flag down. No CERTIFY ran in it, since a passing one raises the flag and a failing one traps. Every CHECK in it passed, since a failing one traps, and each passing CHECK filled one of the table's sixteen entries, so there were at most sixteen. Every COMMIT in it committed a claim the table holds, and the table holds at most sixteen, so at most sixteen COMMITs can be of claims the run hadn't committed before. Sixteen and sixteen make 32, and the only way past it is a COMMIT of a claim already committed earlier in the run. A claim names its counter's version, and the version changes at every write, so a repeat needs its counter left alone since the earlier COMMIT. A guest that writes both counters between any two of its payments can't do that, and it can't commit at all, since a COMMIT needs its counter left alone since its CHECK, so it runs out of ways to pay after sixteen. Without that limit, 32 is reached: sixteen rounds of CHECK *at least 0 A*; COMMIT *at least 0 A*; INC A pay 32 with no trap and the flag down. A guest that writes both counters at every move of the machine it simulates needs a fresh CHECK for each move it pays for, so it can pay for at most sixteen of them. A simulated machine that charges more often than that has to be paid for somewhere, and PAY is where. The 32 isn't a ceiling on either machine: a program

that checks one fact about a counter it never writes can commit that fact again and again, at a cost of one each, with the flag down and no trap, because the counter's version never changes. On the small machine that leaves one counter to compute with while it repeats, which is too few; the host has a counter for every whole number, so on it the repeats cost the computing nothing. A program written for the host's registers doesn't need PAY: it checks one true fact once, about a register it never writes, and pays every later unit by committing that fact again. Those commits overwrite the channel, so at the move where the reading first says yes it runs the chain's CHECK, COMMIT and CERTIFY back to back and pays the rest of that move after the CERTIFY. U_P can do without it on its class, and PAY is what it needs beyond the class. U_P names no register above 80 and its guest can't name a host register, so two sit untouched. Pay the first unit by raising one of them to 1, which reads "zero holds at 0," and checking it; pay every later unit by committing that fact again, which passes because the register's version never moves. A compiled guest runs its chain back to back and pays after it, so the chain's own commitment is what its certificate finds, and its one check leaves the table at two facts. Every payment costs one, the surcharge is unchanged, and the host is the register host with no seventh instruction. Outside the class the trick fails: a guest that pays twice and then certifies with nothing committed traps, and this host, with the pay-fact sitting in its channel, would raise its flag instead. PAY leaves the channel alone, and that is what those guests need. On a host without certification PAY is a necessity: a guest that pays while its reading is down makes the host run a paid move that doesn't raise its record.

Read as a Thiele machine, the host needs no filing at all: PAY is a paid move that never raises the record. Certified, as Section 15.16 pins it down, sorts every paid move into CHECK, COMMIT or CERTIFY, so PAY has to be filed under one of them, and it's filed as a CHECK of the empty claim, a claim that never holds. The filing is for the interface. It isn't what PAY does on the machine: a CHECK that fails on this machine traps, and PAY goes on to the next line. With PAY filed that way the host stays certified, over its own claim language, in which the empty claim holds of no state. PAY costs exactly one, as the third of the four asks of a CHECK; its check never passes, so it can't stand in a chain; and it leaves the table, the channel and the flag alone, so it can't add to a chain or raise the flag either. For the same reason the floor of three for a flag raised from a clean start still holds with PAY present: the flag still needs a CHECK, a COMMIT and a CERTIFY.

The program is U with one more handler: a seventh branch on the guest's opcode, for PAY, that runs the host's own PAY once. Call it U_P; it's 3735 instructions, with 70 paid sites, the 69 of U plus the PAY. The guest it runs is a program of the small machine with PAY added, and that guest is where the presented machine goes. Its four programs are compiled into two counters, by the packing Minsky used, using

the standard translations from computable functions to counter programs; the guest keeps the presented machine's state coded in its counter B, takes one of its moves by running the compiled driver and step, runs the compiled reading on the new state, and pays the move's price in PAYs; the first time the reading says yes, it checks, commits and certifies, and those three paid instructions count toward that move's price. The host's one property is fixed too: a claim is either one of the small machine's three words about a number, or "run reading program r on the coded state v," and the host's checker evaluates that with one fixed routine for every presented machine there is. So nothing in U_P knows any particular presented machine. The machine arrives as data, the way Turing's guests arrived on the tape.

A fixed checker that runs whatever program it's handed and always answers needs saying how, so here is the routine. A claim of the second kind carries a number r. The checker decodes r as a counter-machine routine: its instructions, where they sit, which register holds the answer, which register holds the fuel, and how many registers to load. If r doesn't decode to a routine, the answer is no. Otherwise the checker loads the routine's registers from v, reads a fuel number f out of v too (the fuel register's entry in v's prime factorisation), runs the routine for at most f steps, and says yes exactly when by then the routine has left its code with 1 in its answer register. Every input gets an answer within f steps, so the checker is total, and what it means is that bounded run: yes if and only if the routine halts within the fuel the value carries, with answer 1. That's how it stays clear of the diagonal. A total evaluator E that agreed with every total yes-or-no program on that program's code alone would let $g(n) = \neg E(n, n)$ contradict itself at its own code, as long as E is itself computable. This checker never evaluates a program on its code alone: the fuel comes with the claim's value. The guest compiled from M runs M's reading routine itself and puts at least the step count it took into the fuel register, and at such a value the checker says yes if and only if the reading says yes.

For the fuel rule, take a routine containing just one instruction: increment an answer register initially holding 0, then leave its one-line code. Give it fuel 0 and the check says no: it has not completed. Give it fuel 1 and the check says yes: it has left the code with answer 1. The first no did not mean the routine would never answer yes. It meant "not within this allowance." You can run that distinction with a pebble and one instruction card. The real checker does the same job for any list of instructions, with the allowance handed in alongside the claim.

Theorem 3.7 (One host runs every computably presented machine)**.** Let M be a computably presented machine, so that a driver picks its next move (one run per start), a program decides its reading on every state, and its prices are whole numbers; and let s_0 be a start state. Load U_P with the guest compiled from M's □

presentation and s_0 coded in. What the host carries of M's reading is its latch. Then: (1) U_P halts exactly when M's run stops. (2) For every n such that M hasn't stopped before move n, there's a host time t at which the host's register for the guest's counter B decodes to M's state after n moves, the host's ledger is M's ledger after n moves plus a *surcharge*, and the host's flag is M's latch after n moves; and if M stops at n, such a t has U_P halted as well. (3) The host's flag is up at some time exactly when M's reading is yes at some state of its run. (4) If the host's flag is up at time t, the host's own instructions up to t contain a passing CHECK on a slot, then a passing COMMIT on the same slot with the slot untouched in between, then a passing CERTIFY; and the slot held the claim "reading program r says yes of v" for M's own reading program r, that claim holds, and v decodes to a state of M's run where the reading is yes.

There are two things the theorem doesn't say. First, the host's flag is M's latch. If M's reading says yes at some state and no at a later one, the host's flag stays up, so it says the reading was yes somewhere in the run, and it doesn't say the reading is yes now. Second, what the host re-establishes is the reading: the claim it checked is "M's reading program says yes of this coded state," and that's all. A bare Thiele machine needn't tie its reading to anything true about sorting, proofs or the world outside. If M's reading is meant to say a list is sorted, that it does is a separate correctness theorem about the reading, and the host doesn't supply it. The two universal programs also keep their own guests: U runs the small machine's programs over its three words about numbers, and U_P runs computably presented machines over the universal claim language.

The surcharge comes next, because that's where the ledger stops being exact, and below three units it can't be helped. The host raises its flag the only way the host machine allows, by a CHECK, a COMMIT and a CERTIFY, and those cost three. If M's first move that turns its reading to yes costs at least three, the host pays the difference in PAYs and nothing more is owed. If it costs less, the host pays three there anyway, which is the larger of three and the move's price. The surcharge starts at that move: while M's latch is down it's zero, so a run that never certifies is matched exactly. Whether anything could do better depends on what M paid before. When M's whole ledger at the raise is below three, no program on the host machine can match it, since a raised host flag costs three from a clean start. When M's ledger at the raise is three or more, this compiler isn't always the cheapest. Take a guest whose first move costs two and leaves its reading at no, and whose second costs one and turns it to yes. This compiler carries it at 2 and then 5, against the guest's 2 and 3. A program that looks ahead carries it exactly: inside the first move's price of two it works out the guest's state two moves on, puts it on B, runs the CHECK and the COMMIT of the reading's claim about it, and puts the guest's state back; inside

the second move's price of one it runs the CERTIFY, which needs only a fact in the channel, and nothing has committed since. Its totals are 2 and 3, and its flag stands on the guest's reading at the first state where it said yes. Part (1) of the theorem below bounds this compiler, and part (3) is the only lower bound proved.

Theorem 3.8 (Exact, within two, and no better)**.** Let M, s_0 and the loaded host be as in Theorem 3.7, and let M's reading be no at s_0. (1) For every n before M stops there's a host time at which the state is decoded and the flag matched as before, and the host's ledger is M's ledger after n moves while M's latch is down, and that plus the larger of $3 - c$ and 0 once it's up, where c is the price of the move that first turned M's reading to yes; so the surcharge is at most two. (2) If moreover every move that first turns M's reading to yes costs at least three, the surcharge is zero and the host's ledger equals M's exactly at those times. (3) If M's latch is up after n moves while its ledger is below three, then no run of the host machine from a clean start, under any program at all, ends with its flag equal to that latch and its ledger raised by exactly M's ledger. (4) If instead M's reading is yes at s_0, then at the host time matching each move n before M stops, the host's ledger is M's ledger after n moves plus exactly three. □

Part (3) is an observation in the sense of Section 2: it reads off Observation 1.19, which holds on the host machine with PAY for the reason given above. Part (4) is the same three paid at once: if the reading is already yes at the start, the host has to raise its flag before M has paid anything, and at that point no program can do better, since M's ledger there is nothing. After it the three stay on only because this compiler doesn't count them against M's later prices.

And one machine is carried all the way, by proof, so the class isn't empty. Its states are whole numbers, its one move adds one and costs three, its driver always picks that move, and its reading is "at least three." The driver returns the fixed move; the step adds one; the price returns three; and the reading compares the state with three. Counter programs implement all four. Loaded into U_P from 0, the host never halts, its flag goes up, at the host time matching move n its ledger is exactly $3n$ and its flag is up exactly when $n \geq 3$, and the raised flag stands on a host chain whose slot claims the machine's own reading of a state at least three. That follows from the two theorems above: the reading is no at 0, and the one move that turns it to yes, from 2 to 3, costs three, so part (2) of the surcharge theorem makes the ledger exact.

Now the limits, each of which is in the statement, said plainly.

- **Readings that are decided.** The reading's program has to answer yes or no on every state. A reading a program could only confirm, waiting forever on the no cases, isn't covered, and neither is a reading no program computes at all.
- **One run per start.** The driver picks the move, so the guest is deterministic. A machine whose next move is somebody's unforced choice gets in only with that choice written into its driver.
- **The record is the latch.** The host's flag is up once M's reading has been yes at any state of the run, which is exactly the latch of Section 12. A reading that goes yes and then back to no is carried as "it was yes once."
- **The ledger is exact up to the surcharge.** When the reading starts at no, at most two, and zero when the first raising move costs three or more; when it starts at yes, three. And no program on this host, run from a clean start, can do better when the guest certifies with its whole ledger below three.
- **Prices in whole units, paid one at a time.** A move that costs a million is a million PAYs. The compiler also uses Minsky's packing. Between them, those two choices set how long the host takes to pay for the guest's moves.
- **Proved; never run on a presented machine.** The demonstration machine's numbers come from the two theorems above. U_P itself has run on guests small enough to finish; a presented machine's compiled guest is far too large for that, and nobody has run U_P on one.

CHAPTER 4

Built from parts

Back in Section 2 I asked a question and only got halfway: bolt two Thiele machines together, and do you get one? Everything real is built out of parts. A computer is a processor and memory and a disk, a bank is branches, a proof is lemmas. If the record and its toll only held for one machine at a time and fell apart the moment you put two together, they'd be a property of toys.

So put two Thiele machines on a bench, each box with its own panel, and wire them up the three ways anybody wires machines. Side by side, with no wire between them at all, so that every move is a lever pulled on one box or the other. End to end, with the first box's bins tipped into the second's when the first is done. And one inside the other, with the inner box running as a guest on the outer one. Some of it holds and some of it provably doesn't, and the places where it doesn't were where I learned the most.

4.1 Side by side

Two boxes on the bench, no wire between them. A move of the pair is a lever pulled on one box or the other. The question is what the pair's record should be, and my first answer was the wrong one: a single count of how many certifications the bench has made, one number, easy to read. I'll come back to why it fails, because it's in the list of things that break. What works is to keep both panels showing. The pair's record is the pair of records, a place for the left machine and a place for the right one, compared one side at a time.

Since that record has more than two places, "certified" in this section means the four things of Section 15.16 with the record allowed to be a place along a direction, the way Section 5 set it up. Read on a direction, clean states all sit at one place, the floor; a record never moves below where it was; and in the second of the four a raised record is one that isn't at or below where the run began, with the CERTIFY the move that first takes it out. On two places this is exactly the two-place reading.

□ **Theorem 4.1** (Side by side)**.** (1) The cost of any run of the pair is the left machine's cost of its moves plus the right machine's cost of its moves, and the order in which levers on the two boxes were interleaved doesn't change where the pair ends up or what it paid. (2) If both component state spaces are nonempty, the pair obeys the toll exactly when both machines do. (3) Read through "either flag is up," two Thiele machines side by side are a Thiele machine with floor one; read through "both flags are up," a run that starts with both flags down and ends with both up costs at least two, and two is reached when each part can certify for one. (4) Two certified machines side by side are a certified machine. (5) In that pair, call a side's *floor* the place on its record axis where its clean states sit. A run from a clean start that ends with each side at or above a point that isn't at or below that side's floor makes at least six record moves and raises the ledger by at least six. Six is reached, from suitable loaded starts, by a three-act certificate on each side; a particular pair of target points can need more. (6) The pair of records is the product in the precise sense: for any machine mapping into both, preserving records, there's exactly one map into the pair that does both jobs.

Proof. (1) A move of the pair is a lever of one box. It moves that box's component by that box's step, leaves the other component alone, and costs what that box charges for it. So the pair's run is the pair of the two boxes' runs along the projections of the trace, each box's moves in the order they came, and the cost is the left projection's cost plus the right's. Two interleavings with the same two projections end in the same state at the same cost. (2) The toll prices a step that takes the record out of the down-set of where it stood. A pair is at or below another when both sides are, so the down-set of a pair is the two down-sets side by side, and a move of the left box leaves the pair's down-set exactly when it leaves the left down-set, the right side not having moved. So the pair's exits are the left exits and the right exits, each priced by its own box, and the pair obeys the toll exactly when both parts do; for the converse, an exit at zero cost on one side is an exit at zero cost of the pair with any state at all on the other. (3) Read through "either flag is up," a step from neither to either raises one side's flag, and that side charges it at least one, so the pair obeys the toll with floor one. Neither side's flag ever falls, so under either reading the pair's never does; each side has a named start reading no, and from the pair of those starts the left side's raising run raises "either flag," and the left raise followed by the right raise raises "both flags." So the pair is a Thiele machine under both readings. Read through "both flags are up," a run from both down to both up raises each side's flag on some step, no move touches both, and each such step costs at least one; when each part can certify for one, the two certificates one after the other cost

two. (4) Give the pair the interface that reads a move as its own box reads it, takes a claim to be a claim of one side, about that side's component and unchanged when that component is unchanged, with both sides clean as clean, both sides loaded as loaded, the sum of the ledgers as ledger, and the left box's window as window. Clause (a) is the left box's clause (a), since a left base move acts on the left window as its instruction does and a right move doesn't touch the left component, and no base move of either side changes either record. Clause (c) is the two price lists added. Clause (d) is the left box's witness claim with the right box idle. Clause (b) is the one with work in it. Take a run from both clean that ends with the pair's record raised, and cut it just after the first move that takes the pair's record out of where it began. That move is one side's, and since pairs compare side by side it takes that side's record out of where that side began. Project the cut run onto that side: a run from clean that ends with that side's record raised, so by that side's clause (b) it contains that side's chain, a passing CHECK of a claim c, a COMMIT of c with c's component unchanged in between, and a CERTIFY. Put the other side's moves back where they were. None of them touches the component c is about, so the chain is a chain of the interleaved run. (5) If a side ends at or above a point that isn't at or below its floor, its record ends not at or below its floor either, or the point would be at or below the floor by passing through it. The clean start puts that side at its floor, so the projection onto that side is a run from clean with its record raised, which by that side's clause (b) has three record moves of its own, a CHECK, a COMMIT and a CERTIFY. The two sides' moves are different moves, so six record moves at least, and by (c) a ledger rise of at least six; a three-act certificate on each side, from loaded starts where each side's witness claim holds, makes exactly six. (6) A record-preserving map from a machine Z into a box is a map of states with a monotone map of records, the record of the image being the image of the record, and every move of Z simulated by some run of the box. Given one into each box, send a state of Z to the pair of its images. The record of the pair is the pair of the records, and a move of Z is simulated by the left box's run followed by the right box's, which is a run of the pair by (1). That map followed by either projection is the map you started with, and any map into the pair with those two projections agrees with it on every state, which is what equal maps means here. ∎

Most of that is what you'd guess from the bench. Each box keeps its own ledger and nobody else pulls its levers, so the costs add, and it can't matter which box you happened to work on first. The six in (5) is two certifications costing three each, one per box, with no discount for doing them on the same bench.

Machines, with maps that keep the record, form a category, and item (6) says these pairs are its products. Ask the maps to keep the cost too and the pair stops being a product: a machine mapped into both boxes would pay its cost once on

each side, twice in all. What survives is a symmetric monoidal structure, which is all that side by side needs. Its pentagon, triangle and hexagon laws say that regrouping four boxes, inserting an idle box, or exchanging groups of boxes gives the same final wiring whichever permitted route you take. Label the boxes and follow each wire: every rewiring only moves boxes around, so both routes put each wire at the same labelled terminal, at the same cost.

That paragraph is the most formal way I know of saying "side by side composes, and nothing about the order of the parts matters." It's a lot of category theory to confirm what the bench told me in a glance, and I'm glad to have it anyway. A glance is where I'd have missed something.

4.2 One after the other

Now wire them end to end. Run the first box, and when it's done, tip its two bins into the second box's start, at no charge on the ledger, and run the second. Programs get chained that way everywhere, and my first thought was that the tipping couldn't matter, because it's plumbing: pebbles go from one bin into another, nobody checks anything, and plumbing has no business on a record. That turned out to be half right, and the wrong half is item (4).

□ **Theorem 4.2** (One after the other)**.** (1) Running one certified machine and then handing its two counters to another through loads that cost nothing gives a certified machine. (2) For any two programs of the two-counter machine, it computes the composite: some schedule leaves the first machine's window where the first program takes the input, and the second's where the second program takes that result. (3) The hand-off keeps the toll when every state it can load into sits at one level of the second machine's record. (4) If some load lands the second machine at a record that isn't at or below another load's, the hand-off loses the toll: it's a certification nobody paid anything for.

Proof. (1) Give the second machine a loadable form. A state is either pristine, holding the second machine's loaded state for some (a, b), or running, a state of the second machine. A move LOAD (a, b) turns a pristine state into the pristine state for (a, b) and does nothing to a running one; the second machine's own moves move a running state as before and turn a pristine state into a running one. LOAD is a base move and costs nothing. When the second machine is certified its loaded states are clean and stand at the floor, so a LOAD leaves the record alone; the base, the toll and the witness claim are the second machine's; and the chain in front of a raise lies after the last LOAD before the second machine's first move, so it lifts

through the projection onto the second machine's own moves. So the loadable form is certified, and the composite, the first machine side by side with it, is certified by Theorem 4.1(4). (2) A wired schedule is the first machine's moves, then a LOAD of the two counters its window shows at that moment, then the second machine's moves. The first stretch runs the first program on the first window; the LOAD puts the second window at $(1, a, b)$ for the counters (a, b) the first stretch left; the last stretch runs the second program on it. If both programs halt, the two windows after the schedule are the two halting configurations, the second's on the first's output, and the wire cost nothing. (3) and (4) are one fact read two ways. A LOAD from the pristine state for one input to the pristine state for another costs nothing, and carries the record from where the first loaded state stands to where the second stands. If every loaded state stands at one level, no LOAD moves the record, the composite's exits are the two machines' own, and the toll is kept by Theorem 4.1(2). If some loaded state stands at a record not at or below another's, the LOAD from that other one to it, at zero cost, leaves a down-set, which is an exit nobody paid for, and the toll fails. ∎

Item (4) is the one to remember. The tipping costs nothing, so it had better not tip in anything that counts as established. If pouring pebbles into the second box can land its panel somewhere that isn't at or below where some other pouring would land it, the second machine gets a raised record without ever having checked anything, and that's a forged certificate passed off as plumbing. My first thought was right about the pebbles and wrong about where they land.

4.3 Connecting records and writers

Joining two boxes also means deciding how to read their flags and who may write their workspace. The following examples trace those connections and the guarantees they affect.

Theorem 4.3 (What breaks)**.** (1) Read the pair through "both flags are up" as a single flag, and any run that raises it from a clean start makes at least six record moves; certified asks for a claim that three moves can certify, so read that way the pair isn't certified. (2) Two clocks side by side obey the toll with a two-counter machine inside. No move of the pair costs nothing, so composing that easy bar composes nothing. (3) Give the small machine a second writer that changes counter A without raising its version, and the run "check that A is 0, let the other writer change A, commit, certify" raises the flag, so the earned-record clause fails when the machine is read the small machine's way, with the second writer as a base move. (4) Two windows that each price their own machine exactly can collide as a pair, □

so exact pricing isn't preserved by putting machines together. (5) Count the pair's certifications with one number, and the count can't tell which side certified. The pair of records can.

Proof. (1) Through the pair's interface, a run from a clean start that raises "both flags are up" ends with each side's record raised, and by Theorem 4.1(5) that takes at least six record moves. Certified asks for a claim whose CHECK, COMMIT and CERTIFY, those three moves alone, raise the record from a loaded state. Three moves aren't six. (2) A clock is the two-counter machine with every move priced at one and a reading over it. Two side by side have a two-counter machine running inside, the left's, and obey the toll by Theorem 4.1(2); and every move of the pair is a move of one clock, costing one. (3) Add to the small machine a writer that changes counter A and leaves its version alone, read as a base move. The run CHECK (A is 0), the writer's move, COMMIT (A is 0), CERTIFY, from a start with A at 0, passes its CHECK, finds its fact at the unchanged version at the COMMIT, and raises the flag at the CERTIFY. The base and toll clauses still hold, and so does the checker's promise. What fails is clause (b). The small machine reads "unchanged" as same value and same version, and the only chain in that run has the writer's move between its CHECK and its COMMIT, where counter A, the thing the claim is about, changed its value. The COMMIT didn't notice, because it looks only at the version, and the version didn't move. So a run from a clean start raises the record with no earned chain in it. (4) Take a left box whose only step its window doesn't see is a certifying one, so the window still prices it exactly, charging that unseen transition one; and a right box whose only step is a stutter, at zero cost, leaving its down-set alone and its window unchanged, priced exactly at zero. In the pair, the left box's unseen certifying step and the right box's stutter show the same pair of windows before and after. One costs at least one and leaves the pair's down-set; the other costs nothing and doesn't. So a price computed from the pair of windows is short on the first or over on the second. (5) Replace the pair of records by the number of sides that have certified. A run in which only the left side certifies and one in which only the right side does both reach the count one, with the left record raised in the first and at its floor in the second. The count can't tell them apart; the pair of records can. ∎

Item (1) puts one flag over two boxes. It costs what two certifications cost, six record moves, and the definition wants a claim that three can settle, so a single flag over both panels, read through the pair's interface, is too expensive to meet the definition. That holds for this interface; nothing here shows that no other interface on the pair could meet it, and Theorem 4.1(4) shows the pair meeting it through a richer record. Item (4) says that a window exact for each box alone can be fooled by the pair: two pair states with different costs can show the same pair of readings.

Item (3) is the one with the most teeth, because it's the bug from Section 15, the file checked and then swapped. On the bench it's a second hand reaching into a bin while the box isn't looking: two writers on one value, and a check that's out of date by the time anybody leans on it. The small machine's version rule is exactly the fix, and this is what happens without it: the commitment goes through on a claim about a thing that changed underneath it. Database people and the people who build processors have been fighting this one since at least Kung and Robinson's paper of 1981 and the Livermore report of 1987, and that section gave it the security people's name, time of check to time of use. Here it's a theorem about the record, and the version number is what makes it go away. The second hand can still reach in, but with the version its reach shows on the tally, and the COMMIT traps.

Item (5) is my first guess from the side-by-side wiring, coming back to fail on schedule. It's why the pair's record compares one side at a time. Count only how many sides certified and the record forgets which side did it. That count answers "how many?" but cannot answer "did the left side certify?" An ordinary number could encode the full pair too: use 0 for neither, 1 for left only, 2 for right only and 3 for both. It is the counting rule that loses the distinction, not the fact that its answer is one number.

4.4 One inside another

The last way to wire the bench: put one box *inside* the other, as a guest on a host. Part of this the book already has. The section on the universal machines (Section 3.2) showed the fixed program U_P runs any computably presented machine, and the small machine's own programs, with the record earned again by the host. What's new is putting those links in a row, a box in a box in a box, and asking what the outermost one pays.

My first guess was that it could pay less, even nothing for the guest's climbs, since a host that's clever about how it runs the guest could skip the guest's costs. The first item below puts a floor under that: the host pays at least one for each step where the guest's record went up. It's a floor on rises, and it doesn't keep the guest's whole cost.

Theorem 4.4 (Nesting)**.** (1) Let a host that obeys the toll simulate a guest faithfully: each guest state has an image among the host's states and each guest record an image among the host's records, the host's record at the image of a state is the image of the guest's record there, and each guest move is carried by some host run from the image of the state before it to the image of the state after it. Let the record map also reflect the order: when the image of one guest record is at or below the image □

of another, the first record is at or below the second. Say a run of the host *follows* a run of the guest when it splits into one stretch of host moves for each guest step, each stretch carrying the image of the state before that step to the image of the state after it. Every run of the guest has a host run that follows it, every such host run ends on the image of the guest's end state, and every host run that follows a guest run costs at least the number of the guest's steps that turned a cup over. A host run that only ends on the image of the guest's end state can cost less. (2) In each link the surcharge is 0 until the guest's reading first says yes, and the bounds below hold from then on. A small-machine guest on U_P is carried with surcharge exactly 0. A computably presented machine whose reading starts at no is carried, through its compiled guest, with surcharge at most 2, and, when every move costs at most one, exactly 2 once its reading has first said yes. One whose reading starts at yes is carried with surcharge exactly 3. (3) Links compose, and their surcharges add: a tower of k links has surcharge between the sums of their lower and upper bounds. (4) A tower of presented machines, each level running the next on U_P, adds exactly 2 for every level above the first, on top of the first level's own surcharge; every level's host run is presented by the next.

Proof. (1) The simulation gives, for each guest move, a list of host moves carrying the image of the state before it to the image of the state after it. String those lists together along the guest run: that host run follows it, and it ends on the image of the guest's end state, because each stretch ends where the next begins. Now take any host run that follows the guest run, and a guest step that turned a cup over, leaving the down-set of its record. Its stretch carries the image of the state before to the image of the state after, and since the record map reflects the order, the image after isn't at or below the image before, or the guest's record after would be at or below the record before. If every host move in that stretch ended at or below where it started, then chaining them, the end of the stretch would be at or below its start; so some host move in it ends not at or below where it started, leaving the down-set of where it stood, and the host, obeying the toll, charges that move at least one. Different guest steps have different stretches, so the charges add up to at least the number of such steps. The end alone isn't enough: a guest on one cup that turns it over and turns it back has one rise and ends where it started, and the host run that does nothing reaches the image of the end state at no cost. (2) A link from a guest to a host matches every point of the guest's run with a point of the host's, with the record bits equal there and the host's ledger the guest's plus a surcharge, the surcharge 0 while the record is down and between the link's two bounds once it's up, and halting and the record's first rising matching. A small-machine guest run by U_P is a link with surcharge exactly 0: that is the ledger equality of Section 3.1, carried to U_P. A computably presented machine run as its compiled guest is a link

with surcharge at most 2 from a reading that starts at no and exactly 3 from one that starts at yes, which is Theorem 3.8; once the reading has first said yes the surcharge is the larger of $3 - c$ and 0, with c what the raising move cost, since the host pays the larger of c and 3 there, and when every move costs at most one, c is exactly one, the toll making it at least one, so the surcharge is exactly 2. (3) Match the points through the middle: a point of the bottom run has a matching point of the middle run, which has a matching point of the top run; the record bits agree along the way; the top ledger is the bottom ledger plus both surcharges, which is 0 while the record is down and between the sum of the lower bounds and the sum of the upper bounds once it's up; halting and the first rising pass through. A tower of k links is that, $k - 1$ times. (4) Each level above the first is, by the presentation after this proof, a presented machine whose run is the level below's host run, carried as its compiled guest on U_P. Its moves are the host's moves, which cost 0 or 1, so by (2) that link adds exactly 2 once the record is up, and by (3) the additions stack on the first level's own surcharge. ∎

Item (1) is the cleanest sentence in the appendix. However you build the outer box, as long as it obeys its own toll and doesn't lie about the inner box's record, it pays at least one for every step where the inner record climbed, on every run that keeps pace with the inner box step by step. The keeping pace is needed. A guest that turns a cup over and then turns it back ends where it started, and the host can reach that image by doing nothing at all, at no charge on its ledger. The step by step floor is the guest's rises paid one by one, and the end alone can't see them. It's a floor on the climbs, and it isn't the inner box's whole cost. A guest can charge a hundred for one rise while a host charges one for the matching rise, and both obey their tolls. Keeping the whole cost takes a further demand on the simulation, that it carry the guest's costs too, and for U_P that's the separate ledger theorem of Section 3.2, with its surcharge.

Part (4) looks as if it needs a premise, and it doesn't. Two links are plainly unconditional: from a presented machine to its compiled guest, and from that guest to U_P running it. Going deeper needs the host's own run, at each level, to be presented as a machine the next level up can run. All the host's states together have no one-to-one code in one number, since there's a register for every whole number. But a loaded state has only finitely many registers that aren't at 0 with version 0, and no move changes more than one of them, so the states like that are a machine of their own, closed under every move, and each has a code: its line, its finitely many live registers with their versions, its table, channel, trap, ledger and flag. Programs that always answer compute its step, price and flag from that code, and the driver reads U_P's instruction off the line and says stop when the host has stopped. Its run is the host's run, and its toll is the host's. So every level is presented by the next,

and the tower adds 2 a level as deep as you like. The 2 comes from the compiler's rule of stopping at zero; a compiler that carried the excess forward would add 2 at that level's raise and give it back as the level below paid on.

4.5 What this adds up to

Wired side by side, end to end, or one inside the other, the toll carries over from the machines on the bench, and the tolls add the way money adds. What breaks, breaks for reasons you could name before you saw the proof: a single flag for two claims costs two claims' worth, a clock is still a clock, a shared value without a version can't be trusted, and a hand-off that costs nothing can't be allowed to carry a certificate. None of those is a surprise in hindsight. All of them are now theorems, and the composition question from Section 2 has an answer. Two Thiele machines side by side are one under both readings I tried, "either flag" and "both flags." The "both flags" reading still fails certified through the pair's interface, the second writer breaks the earned-record clause through the small machine's own reading, the clocks and the two counterexamples fail outright, and simulation in general has a proved floor.

Three of my first guesses on this bench were wrong: the single count, the harmless tipping, and the clever host that pays nothing for a climb. The last two leaned toward a record that says more than anybody paid for, and the first toward one that says less than it should. I didn't see any of the three coming; the proofs did.

CHAPTER 5

The toll on this machine

This machine runs the book's eight-state machine. Seen through a window that shows only a line number, read modulo four as the slot, and a flag, a short program of that machine, once a two-instruction setup has run, does what the eight-state machine does, one move at a time, and pays exactly what the eight-state machine charges. The example as a whole prices its record and not every merge: some of its zero-price steps merge states, and whatever those merges cost the hardware is off the ledger, so the finite piece is a piece. The toll on the whole example comes from the record's part of the state (Theorem 5.1).

Finiteness did one job in the counting. It forced the step that raises a permanent flag to merge two states. On this machine no count over the whole machine has to force that, because the merge is in plain view. Its record moves are the check, the commitment and the raised flag that the opening's lock puts in order, and it names them CHECK, COMMIT and CERTIFY: COMMIT names a checked claim in a slot called the channel, and CERTIFY raises the flag. CERTIFY takes a state with the flag down and the same state with the flag up to one place, since a flag that's already up just stays up. Its COMMIT overwrites the channel, so two states that differ only in what the channel names land on one state. Part (1) of the theorem below shows both. So any price list that charges every merge of that machine's own step charges the flag-raising step, with no premise that the machine is finite. It isn't: its versions and its ledger grow without bound.

Finiteness is still there, in the smallest place it can be. Hold every part of the state but the flag fixed, and the flag has two readings, down and up. CERTIFY sends both to up, and it does the same to everything else from either, so the two states land together. That is the counting of Theorem 3.1 on a set of two. So the choice I made in the opening's list isn't the merge, and it isn't the shape of CERTIFY. It's that the step never reads the flag: the rest of the state moves by its own rule and the ledger adds the instruction's price, whichever way the flag reads, so no instruction keeps a copy of the old reading, and any that raises the flag lands its two readings together. Make CERTIFY trap instead when the flag is already up, and the merge moves rather than goes away: the state it traps lands with the state that was trapped

already, since a trapped machine only pays. That's the halted state stepping to itself again, and it holds of every record move: any of the three that fails lands with the same state already trapped. The one way out of the flag's merge is to write the old reading into a part of the state that grows, a list of every reading so far, say, and that is a state never used before, the other column. A ledger that adds more for a re-raise than for a raise doesn't do it: the state with the flag up and the ledger a little lower lands where the raise lands, and the merge has only moved again.

That leaves the other miss, the unpriced merges. This machine's price list puts its flat moves at zero, and a flat move merges (DEC, counting a counter down, lands two states on one), so full merge pricing and that price list can't both hold on that machine. They don't have to. Split a state into its base part, which is what a two-counter machine would show (the line number and the two counters), and its record part, which is everything else: the versions (each counter's count of how many times it has been written), the table of facts, the channel, the trap latch (which goes up when a record move fails, after which only the ledger moves), the ledger and the flag. Then price only the merges the record layer makes.

Premise 5.1 (Record-layer merge pricing)**.** A move that sends two different states with the same base part to one state costs at least one.

Part of where that line falls is forced. The line number has to be in the base part, or DEC, which can land two lines with the same counters on one, makes a record-layer merge at no cost; each counter has to be there, or its version, or DEC's own merge gets charged; and the flag or the trap has to be outside it, or nothing charges CERTIFY. Where the versions, the table, the channel and the ledger sit moves nothing below.

The merges among base parts are the flat part's business, the ones any two-counter machine makes, and whatever heat they cost is the flat part's heat. The premise prices only merges between states with the same flat part. A step like DEC, which changes that part, can lose a version number at no cost, and the premise doesn't object. Merge pricing implies this premise, so it asks less, and on the record distinctions it is still an assumed operation-level price; identifying it with heat needs a physical model. And it is a price on this machine, not a source of the toll on any machine. A certifying move whose effect on the base read the flag, say one that moved a counter up when it raised the flag and left the counter alone when the flag was up already, would raise the flag and send no two states with the same base to one state, and the premise would have nothing to charge at the raise. It would still merge: the state with the flag down and the state with the flag up and that counter one higher land together, across two base parts, where the premise doesn't look. And on this machine its trap would still be charged when it fails. The

small machine's record moves treat the base the same whichever way the flag reads, which is why the premise does its job there, and the theorem below is stated for that example and no other.

Theorem 5.1 (The toll on the example machine)**.** For the small machine of Section 1, with no finiteness premise: (1) CERTIFY merges, and so do COMMIT and CHECK of every claim, and each joins two different states with the same line and counters. (2) Every cost that satisfies merge pricing for that machine's step charges CERTIFY, every COMMIT and every CHECK at least one, and obeys the toll; its flag never comes down and a run from a clean start raises it (Theorems 1.6 and 1.20), so the machine with that cost is a Thiele machine. (3) No INC, DEC or HALT sends two different states with the same line and counters to one state. So the machine's own prices, base moves at zero and each record move at one, satisfy Premise 5.1, they're the least prices that do, and merge pricing fails for them. (4) Every cost that satisfies Premise 5.1 obeys the toll, so, by the same two theorems, the machine with that cost is a Thiele machine. □

Proof. (1) For CERTIFY, prepare two states identical except for the flag. Give both a channel holding a commitment. One flag is down and one is up. CERTIFY raises the first and leaves the second up. Both advance one line and add the same charge; every other field agrees. They have become the same full state.

For COMMIT, put the requested fact, at the counter's current edition, in both tables. Leave one channel empty and put that very commitment in the other. COMMIT writes the same commitment into both channels and treats every other field alike. Again the full successors agree.

For any CHECK, fill the fact table to its sixteen-entry limit. CHECK then fails, whatever claim it was asked to test. Compare that untrapped state with an otherwise identical state whose trap is already up. The first sets its trap; the second is already frozen. Neither changes the other core fields, and both pay the same charge. Their successors agree. In each construction the starting line and counters agree as well, so these are record-layer merges.

(2) A price on every merge charges each of those instructions at least one. Only CERTIFY can raise the certificate. Its merge therefore supplies the toll.

(3) Check the free instructions by asking whether two states with the same line and counters could become one. The table, channel, trap, ledger and certificate are left alone, so any difference in those fields survives. On a live state the shared line and counters choose the same instruction branch. A counter write adds one to its edition; different edition numbers stay different after the same addition.

All unchanged editions stay different too. Hence no remaining field can hide a difference between those sources.

A trapped state stays exactly where it is under a free instruction. Two different trapped states therefore stay different. A live state remains live under these instructions, so it cannot meet a trapped one. That exhausts the cases. The free instructions make no record-layer merge, and all three kinds of record instruction do. Charging the former zero and the latter one satisfies the premise and is the smallest such price list. Full merge pricing still fails: the zero-price DEC pair in Theorem 1.22 starts with different counters and lands together.

(4) Any record-layer merge price charges the CERTIFY pair from (1). Since no other instruction raises the flag, this gives the toll for every raise. ■

So this section reaches the example machine. On it the toll follows from record-layer merge pricing, and the example's own price list, flat moves at zero included, satisfies that premise. This is a result about its ledger, not a heat bound. There's a second way to settle the clash, which I note and didn't take. Morita's reversible two-counter machine runs every step backwards and is still universal: with its input encoded, it computes anything a Turing machine can. Put the record over a base like that, with the base's step taken as the program's own step from line to line and the halt left with no next step, and every base step is one-to-one, and full merge pricing permits those base steps to cost zero. That price list would put the flat moves at zero; the lower-bound premise alone does not require zero charges. It isn't the example's flat part, though. The example has a move for each two-counter instruction, acting from every live state, not a step for each line of one program, and DEC $c\ j$ sends states on any two lines with the same counters, c above 0, to the same window, so whether it merges turns on the rest of the state and not on the program being reversible. That base wouldn't be getting anything for nothing: what it doesn't forget it keeps in its counters, which is the column of states never used before, again. I haven't built that machine.

CHAPTER 6

No bare verifier on this machine

On this machine the collision is a theorem too, and it doesn't depend on the two-move illustration. On that machine computing costs nothing and never touches the record, and from a clean start the record goes up only on an earned check, commitment and certificate. Give a run from a loaded start (Part III's word for a start at line 1 with the two counters set to a and b, written $(1, a, b)$) the *bare transcript* made of a, b and the window on the counters the run ends with, and say a state *explains* a bare transcript when some run from that start ends at it showing that window.

Theorem 6.1 (No bare verifier on the small machine)**.** On the small machine: (1) No function from bare transcripts to Booleans is both sound and complete, under that explanation relation, for the claim "the record is up." (2) There is a clean loaded state such that no function from bare transcripts to Booleans is both sound and complete for the claim "the ledger is at least 3 above what it was at that start." (3) For this machine, whose moves CHECK, COMMIT and CERTIFY and whose counter move DEC are defined in Section 1: a run CHECK *is zero* A, COMMIT *is zero* A, CERTIFY from the loaded start $(1, 0, 0)$, which ends with the ledger at 3, and three DECs of a zero counter from the same start, which end with the ledger at 0, end with the same window, and no function on bare transcripts is sound and complete for "the ledger reads 3." □

Proof. Theorem 1.15 gives a clean loaded state and two runs from it that end with the same window, one with the record up and one with it down, one whose ledger rose by at least 3 and one made of counter moves at no cost that leaves the ledger where it started. Both end states explain the same bare transcript. A verifier complete for the claim accepts that transcript, since a state where the claim holds explains it; a verifier sound for the claim then makes the claim true of every state that explains it, including the one where it's false. That's (1), and (2) is the same with the ledger: the paid run's end state satisfies the claim, and the unpaid run's doesn't. For (3) the two runs are written out in the statement: three moves land

on line 4 in both runs with both counters at 0, and the same two-state argument applies. ■

Put with the three escapes, the whole picture holds for the small machine: the bare-transcript verifier doesn't exist; the collision survives into any richer transcript that keeps it, so no verifier that only reads its bare part works either; a verifier on full states exists; one on the bare transcript with a committed bit exists, sound and complete when the bit is honest, and the unchecked bit breaks the contract; and one on a truthful response exists.

Parts (1) and (2), with a loaded state for a clean one and "at least 1" for "at least 3", hold of any machine whose flat moves cost nothing, can reach every window and can't touch a record that some run raises, by the same argument; certification is what raises the gap to three.

CHAPTER 7

Scope of these results

Who it's safe against. Every guarantee about the record is about programs: any list of instructions, run from a clean start on a step that does what its definition says, one writer stepping at a time. That covers the buggy program and the hostile one. It doesn't cover whoever sets the start (a snapshot put back, a table loaded with facts nobody checked), a reset or a power cycle, the wire that carries the report, someone who flips the flag with a probe, the tools that built the step, or a second writer that changes a counter without bumping its edition. Those are the operator, the device and the tool chain, and they're outside this fence. A record read by someone who didn't watch it being made is only as good as their reason to believe the box that shows it. The ways across that I know of, a quote over measured state and a proof that travels on its own, are in Section 9 and Section 16, and this book builds neither. What the guarantee gives inside the fence is safety: no program forges the chain. Availability is a different promise, and it isn't given. A hostile program can put the trap up or fill the table, and after that nothing useful gets done. The mathematical machine uses whole numbers, which never overflow, so versions and the ledger grow without bound. A build with fixed-width counters, versions or ledger has to stop before any of them wraps, failing safe with the trap up, because a wrapped version could make an old fact look live again and a wrapped ledger would fall. Starting, restoring, resetting and that overflow stop belong to the implementation, and none of them is proved here.

The diagonal on the machine. What's proved for the small machine's programs is that halting is undecidable in the reduction sense of Theorem 1.5, for them and for the universal program (Section 1; for the universal program, Section 3.1). The wall of Section 14 is stated over any substrate with a recursion theorem and a representable flip, and uses neither the counter nor the toll.

The machine. This companion has one small machine, with the hosts that run it, and this paragraph is its fence. The small machine of Section 1 (two counters, six instruction kinds) is where commitments are earned: from a clean start, a raised flag went through a passing check, a commitment and a certification of the same claim, nothing is forged, and the cheapest such run adds exactly three to the ledger,

a price reached by a check that could have failed (the step doesn't police trivial checks: a check that can't fail certifies for three too). It's also where universality is proved, by matching Minsky's two-counter machines step for step and using the halting reduction of Forster, Larchey-Wendling and coauthors. It's certified, in the sense Section 15.16 pins down, and it hides its record and ledger from a window on its counters. The universal Thiele machine for the small machine's programs (Definition 3.1) is a host with the small machine's instruction kinds and more counters, running one fixed program, U, with no guest step built in: it runs every program of the small machine from every clean start start(x, y), halts when the guest halts, raises its own flag by its own check, commit and certificate if and only if the guest's flag rises somewhere in the guest's run, and keeps a ledger equal to the guest's at every matching point. With one paid move added, one fixed program, U_P, runs every computably presented machine: its step, prices and reading computed by programs, the reading decided on every state, the run fixed by a driver. The host halts when the guest stops, its flag is the latch of the guest's reading, raised on its own earned chain whose claim is that reading, and its ledger is the guest's plus a surcharge of at most two when the reading starts at no, a surcharge no program on that host, run from a clean start, can avoid when the guest certifies with its whole ledger below three. A machine whose reading no program decides is outside. The small machine's check price is flat, its fact table holds sixteen, and its certificate is historical (Section 1 lists the rest).

CHAPTER 8

Words for this machine

These words describe this machine and its universal programs, and none of them is part of the definition of a Thiele machine. A *two-counter machine* is Minsky's: two counters holding whole numbers with no upper limit, and two instructions: add one, and subtract one and jump if the counter is above zero, or else go on to the next line; already enough that nobody can decide in general whether one halts. The small machine is a two-counter machine plus HALT and three instructions for the record, six in all: INC, DEC, HALT, CHECK, COMMIT and CERTIFY. A counter's *version* is its edition number: it starts at 0 and goes up by one every time the counter is written, so an edition never comes back. A *fact* is a property, a counter and a version together, such as "A is even, as of edition 3"; the properties come from a fixed language with three forms, *is zero*, *is even* and *is at least* n. The same theorems also hold for any property language with an exact equality test whose checker is proved to say yes exactly when the claim holds, and one such language reads a counter as a list and asks whether it's sorted. The *fact table* holds up to sixteen facts, and the only way in is a CHECK that passed; nothing ever leaves it. The small machine's *channel* holds one committed fact or nothing, and COMMIT puts a fact there only if the table has it about the counter's current edition. With the flag up from a clean start it names a commitment earned that way, and a later COMMIT replaces it, as a two-commit run shows. The *trap* is a bit that goes up when an instruction refuses, and then nothing in the core moves again. A commitment is *earned* when a passing check put its fact in the table first, about the same edition, with no write to that counter in between; on the small machine every flag raised from a clean start is earned, and that's a theorem. CHECK, COMMIT and CERTIFY cost one each and everything else costs nothing, so from a clean start a run ending with the flag up adds at least three to the ledger, and a run whose check could have failed adds exactly three: CHECK *is zero* on A, COMMIT, CERTIFY, from a start where A is 0. The step doesn't police trivial checks; *is at least 0* passes every value and certifies for three too. The *host machine* is the small machine with a counter, called a *register*, for every whole number, and one property, Slot, true of a number that packs a property of the small machine together with a value it holds of. A *slot* is a register of the host used for

nothing but holding one checked fact about the guest, and its *mirror* is a register beside it, safe to read, that says which claim the slot holds while its fact is live. U is the fixed program of host instructions that runs every small-machine program; it reads guest instructions out of one register, keeps the guest's counters in two others, and puts every checked fact in a slot, because on this machine reading a counter above 0 into a program's control flow writes it. The universal program's *guest* is a program of the small machine, held as one number in a register of the host and run one instruction at a time by U's fetch and decode. U_P is U with one more handler, for PAY, an instruction that costs one and does nothing else; its guests are programs compiled from computably presented machines (Section 3.2).

CHAPTER 9

The premises this machine adds

The book's appendix lists every premise its own results use. These three are used only here.

Premise	What it says	Where it's used
Computably presented	four programs compute the driver, step, price and reading on valid codes; the reading program also halts and returns false on junk codes	the universal host for presented machines, Section 3.2
Clean start	an empty fact table, an empty channel and the flag down	every provenance theorem about the small machine, Section 1; the floor of three needs less, and that section says exactly what
Record-layer merge pricing	a move that sends two different states with the same base part to one state costs at least one	the toll on this machine, Theorem 5.1

CHAPTER 10

What's mine about this machine

10.1 An example machine that meets the definition

Known: two-counter machines and the undecidability of their halting are Minsky's (1961, 1967), developed further by Forster, Larchey-Wendling and coauthors (2019, 2020). The guarantee about every program is older than any of this, and it isn't mine either. It's the reference monitor (Anderson, 1972): a check that sits in the path of every operation, so no program gets around it. Schneider (2000) worked out what a monitor that watches a run can enforce: only safety properties, the kind a bad run breaks at some finite moment, and "no CERTIFY without an earlier earned COMMIT" is one of those. Azevedo de Amorim and coauthors (2015) verified monitors that check tags at every instruction. The shape of "earned," a certificate nothing can make except through the rules, is Milner's LCF, where a theorem is a value only the logic's rules can build (Gordon, Milner and Wadsworth, 1979; Gordon, 2000); the small machine is that architecture at the level of single instructions. "No certify without an earlier commit" is also a correspondence assertion in the sense of Woo and Lam (1993), which Gordon and Jeffrey (2003) check with a type system, and a store of commitments that the rules of a dialogue keep, each speaker's updated by the rules as the moves are made, is Hamblin's (1970, chapter 8, on formal dialectic); the difference from all three is again that here the step enforces it. A check that blesses a value and a write that takes the blessing away is close kin to dynamic taint tracking, whose labels go back to Denning's lattice model of information flow (1976), with the blessing itself, controlled endorsement, given a type system and a security theorem by Cecchetti, Myers and Arden (2017), and what a monitor can enforce with bounded memory is Talhi, Tawbi and Debbabi's question (2008). A small monotone record inside each replica of a replicated service is A2M (Chun and coauthors, 2007), TrInc (Levin and coauthors, 2009), MinBFT (Veronese and coauthors, 2013) and Damysus (Decouchant and coauthors, 2022), and Clement, Junqueira, Kate and Rodrigues (2012) worked out, with proofs, what such a record buys and what it doesn't. The signature route to the same goal, which needs no trusted step, is IA-CCF (Shamis and coauthors, 2021): replicas sign and log their protocol messages in the ledger, so blame lands on individual members however

many of them misbehave. And "check once, issue a capability the system trusts, rely on it later" is built and deployed in Garg and Pfenning's proof-carrying file system (2010), whose logic lets a policy's consequences change with time and state, and in Sirer and coauthors' logical attestation for the Nexus operating system (2011), unforgeable statements about programs used as credentials; those are the closest systems I've found to check, commit, certify. Mine: one worked example, a machine of six instruction kinds on two counters, proved to meet the definition so that the definition is not empty, with a version for each counter, a bounded fact table and a channel; its theorems that, from a clean start, a committed fact was checked first about the same edition, that nothing is forged, that nothing is evicted, that with the flag up from a clean start the channel names a fact an earlier passing COMMIT committed, and a later commit can replace it, as a two-commit run shows, and that a run from a clean start to a raised flag adds at least three to the ledger, with three reached by a run whose check could have failed (the step doesn't police trivial checks, so one that can't fail costs three too); the same theorems for any property language with an exact equality test whose checker is proved to say yes exactly when the claim holds, a sorted list read out of a counter included; and the step-for-step match with the two-counter machine, so halting is undecidable for it (Section 1). Kind: an instance of monitor enforcement with two things on top that I didn't find put together where I looked: the check is bound to the edition of the thing it checked, and a toll is paid for the commitment. The first of those alone is old: checking at commit time that nothing you read has changed since is optimistic concurrency control (Kung and Robinson, 1981), and each counter's version here is that scheme's version number. What I didn't find is that check put together with a toll. The universality and the undecidability are borrowed, and the other properties are short consequences of the step rule. Whether the design is new I can't say, and I make no claim beyond the proofs.

10.2 The universal host for the example

Known: Turing's universal machine (1936), which runs any machine given as data, and the universal register machines that run two-counter programs. Keeping a guest and the code running beside it out of each other's state is what a virtual machine monitor promises (Popek and Goldberg, 1974). Compiling computable functions into counter machines is Forster, Larchey-Wendling and coauthors' work. Mine: one fixed program, U, written in the small machine's own six kinds of instruction on a host with a register for every whole number, with no guest step built in, that runs every program of the small machine from every clean start start(x, y), halts exactly when its guest does, raises its own flag by its own check, commit and certificate if and only if the guest's rises somewhere in its run, and keeps a ledger equal to

the guest's at every matching point; its host machine is certified, and whether U halts on a loaded guest is undecidable (Section 3.1). Two small facts force its shape, and they're stated there: reading a counter above 0 into the program's control flow on this machine writes it, and a finite program can't keep the guest's facts on a copy of the guest's counter, by the pigeonhole. The separate host with a built-in guest step belongs to the wider development and is not constructed in this edition. Code that shares unprotected memory with a record can overwrite it, and ordinary computers isolate memory and enforce rules with monitors too, so what I claim is narrower: the simulation and the record guarantee proved for this one fixed program U. Kind: an implementation, with its proofs, of a design whose shape is Turing's universal machine, with the record and the toll carried through. The universality is borrowed from the small machine and from Minsky. And the same program with one paid move added, U_P, runs every computably presented machine, with its halting matched, its reading carried as a latch on the host's own earned flag whose claim is that reading, and its ledger to within a surcharge of at most two when the reading starts at no (exactly three when it starts at yes), which no program on the host, run from a clean start, can avoid when the guest certifies with its whole ledger below three (Section 3.2); the counter-compilation construction is theirs. Kind for that part: an implementation with its proofs again. That a universal machine runs any computable machine is Turing's; what's carried through here is the record, earned again, and the prices, exact up to the surcharge, and I didn't find that put together where I looked. In the words of Definition 3.1, U is a universal Thiele machine for the small machine's programs with no surcharge, and U_P is one for every computably presented machine with a surcharge of at most two when the guest's reading starts at no and three when it starts at yes.

CHAPTER 11

The claims about the worked machine

This companion built one small machine and the hosts that run it, and this ledger keeps them apart from the model. The machine is the small one of Section 1, two counters and six instruction kinds, where commitments are earned and universality is proved, with the universal program U on a host made of the same instruction kinds, and the priced program U_P that runs every computably presented machine. Every line in the next tables is about those machines, or about the machines built from parts, and nothing else, under the premises printed in its section. The book's claim ledger lists the claims about the model.

Claim	Kind	Home
Checker and arithmetic meaning agree	proved on the small machine, under the stated premises	Section 1, Lemma 1.2
Two-counter simulation and halting correspondence	proved on the small machine, under the stated premises	Section 1, Theorem 1.3 (step), Theorem 1.4 (halting)
Halting is undecidable by the two-counter reduction	proved on the small machine, under the stated premises	Section 1, Theorem 1.5
Exact ledger, permanent certification and toll	proved on the small machine, under the stated premises, directly from the rules	Section 1, Theorem 1.6
A Thiele machine	proved on the small machine, under the stated premises	Section 1, Theorems 1.7, 1.6 and 1.20
Certification factors as a latch over the core base	proved on the small machine, under the stated premises	Section 1, Theorem 1.8

Claim	Kind	Home
A passing commitment has a prior matching check at the same version	proved on the small machine, under the stated premises	Section 1, Theorem 1.10
Certification has a check, commitment and certification in that order	proved on the small machine, under the stated premises	Section 1, Theorem 1.11
Unearned commitment and uncommitted certification trap	proved on the small machine, under the stated premises	Section 1, Theorem 1.12
Current-version checked claims are true	proved on the small machine, under the stated premises	Section 1, Theorem 1.13
Equal computational windows can hide different records	proved on the small machine, under the stated premises	Section 1, Theorem 1.15
The window cannot recover the ledger, flag or commitment permission	proved on the small machine, under the stated premises	Section 1, Theorem 1.16
From a clean start, a run ending with the flag up adds at least three to the ledger	proved on the small machine, under the stated premises, directly from the rules	Section 1, Observation 1.19
That minimum of three is reached by a run whose check could have failed, and when that check fails the program is refused forever	proved on the small machine, under the stated premises	Section 1, Theorem 1.20
The step refuses trivial checks	not claimed; “is at least 0” passes every value and certifies for three too	Section 1

Claim	Kind	Home
From a clean start, provenance, checker soundness, no forging and the floor of three hold for any property language with an exact equality test and a checker proved to say yes exactly when the claim holds, a sorted list read out of a counter included	proved on the generic machine	Section 1
With the flag up from a clean start, the channel names a fact an earlier passing COMMIT committed, and a later commit can replace it	proved on the small machine	Section 1
The guarantees hold against whoever sets the start, a reset, or a second writer that doesn't bump versions	not claimed; they are about programs from a clean start, one step at a time	Section 1, Section 21
A projection that hides every CHECK and gives each free instruction its own rule on the picture makes every free move stand still	proved on the small machine	Section 1, Theorem 1.1
On clean runs the flag is exactly the nonempty reading of the publication archive	proved on the small machine	Section 1, Theorem 1.9
No stored fact is forged on a clean-start trace	proved on the small machine, under the stated premises	Section 1, Theorem 1.14

Claim	Kind	Home
Same final counters and bill, different certification: two clean starts and one word end on $(0, 0)$ with ledger 3, one certificate true and the other false	proved on the small machine	Section 1, Theorem 1.17
Exactly 65,535 publication histories can lie above one reachable native endpoint; sixteen extra bits encode them and fifteen do not	proved on the small machine	Section 1, Theorem 1.18
CERTIFY merges and is priced; DEC A 2 merges and costs nothing, so not every merge of the small machine is priced	proved on the small machine	Section 1, Theorem 1.22
A counter beside a Turing machine changes nothing	proved	Section 1.11, Theorem 1.21
A Turing machine can't price a reading	not claimed	Section 1.11
No fixed host program can keep the guest's facts on an exact copy of a guest counter	proved, for any host property language and any translation into a finite list of host properties	Section 3.1, Theorem 3.1

Claim	Kind	Home
A universal Thiele machine for a class: one host and one program, any guest loaded into a start state, halting matched, the guest's states decoded in order, the host's flag the guest's reading latched, the host's ledger the guest's plus a surcharge, never negative and bounded for the class, and the flag raised only by the host's own moves under its own toll	definition (surcharge 0 for U; for U_P at most 2 when the guest's reading starts at no and exactly 3 when it starts at yes, so 3 over the class)	Section 3.1, Definition 3.1
One fixed program U, in the small machine's own instruction kinds and with no guest step built in, runs every small-machine program from every clean start start(x, y), step for step, with halting and answers matched	proved on the host machine	Section 3.1, Theorem 3.2, Theorem 3.3
U's flag rises at some step if and only if the guest's does, and its ledger equals the guest's at every matching point	proved on the host machine	Section 3.1, Theorem 3.4
U's raised flag stands on U's own check, commit and certificate on a slot, and on the guest's own earned chain for the same claim	proved on the host machine	Section 3.1, Theorem 3.5

Claim	Kind	Home
U's host is certified, and whether U halts on a loaded guest is undecidable	proved on the host machine; undecidable in the reduction sense of Theorem 1.5	Section 3.1, Theorem 3.6
One fixed program U_P, U with one paid move added, runs every computably presented machine: halting matched, the state decoded at every matching point, the host's flag up exactly when the guest's reading has been yes, and the host's ledger the guest's plus a surcharge	proved on the priced host	Section 3.2, Theorem 3.7
U_P's raised flag stands on its own check, commit and certificate on a slot whose claim is the guest's own reading of a state where it said yes	proved on the priced host	Section 3.2, Theorem 3.7
The surcharge is at most two when the reading starts at no, and zero when the first raising move costs at least three	proved	Section 3.2, Theorem 3.8
No program on the host machine, run from a clean start, exactly matches a guest certified with its whole ledger below three	proved, for every program from a clean start, directly from the rules	Section 3.2, Theorem 3.8
The priced host is certified, with the paid move read as a check that never passes	proved	Section 3.2

Claim	Kind	Home
A demonstration machine (one move of price three, reading “at least three”) runs on U_P, with the host’s ledger exactly $3n$ at the host time matching move n	proved; never run	Section 3.2
A machine whose reading no program decides, or whose next move no program computes, runs on U_P	not claimed; outside the class	Section 3.2
A guest’s reading that goes back to no is carried as no	not claimed; the host’s flag is the latch of the reading	Section 3.2

Claim	Kind	Home
Products and sequential hand-offs preserve the stated structure; shared writers and unsuitable record readings can break it	proved under the stated interfaces and loading conditions	Section 4, Theorems 4.1–4.3
A following host run pays for the guest’s record rises; presented nesting surcharges add	proved; every host run is presented on its states with finitely many live registers, so towers go to any depth	Section 4, Theorem 4.4

Claim	Kind	Home
On the small machine CERTIFY, COMMIT and CHECK merge, every merge-priced cost obeys the toll, and record-layer merge pricing holds for its own prices and gives the toll, with no finiteness premise	definition; proved (premise: record-layer merge pricing, for the toll)	this companion, Premise 5.1, Theorem 5.1
On the small machine, no verifier on bare transcripts is sound and complete for the record, or for a ledger rise of 3 from a clean loaded state	proved	this companion, Theorem 6.1